

따라하며 배우는

빅데이터 분석

박동규 지음

Big Data
Analytics



06 시본 알아보기

CONTENTS

- 6.1 데이터 사이의 관련성을 알아보자
- 6.2 상관계수를 넘파이로 표현하기
- 6.3 상관계수의 시각화와 해석
- 6.4 간단한 시본 튜토리얼로 시작하기
- 6.5 tips 데이터의 구조
- 6.6 산점도 그래프로 관계를 상세하게 나타내보자
- 6.7 변수 사이의 관계를 알아보기에 편리한 쌍 그래프
- 6.8 FacetGrid 알아보기
- LAB 6-1 groupby()를 이용한 그룹핑과 시각화
- LAB 6-2 배열의 형태를 알아내고 슬라이싱하여 연산하기
- 6.9 사례 분석: 시본의 다양한 데이터 셋과 펭귄 데이터 셋
- 6.10 펭귄 데이터 셋의 시각화
- 6.11 펭귄 데이터 셋의 전체 구조를 파악하고 질의를 하자



06 시본 알아보기

CONTENTS

- 6.12 사례 분석: Anscombe's quartet 데이터 셋
- 6.13 비선형 함수를 사용하여 데이터를 설명하자
- 6.14 사례 분석: 항공기 이용 승객 데이터 셋
- 6.15 항공기 이용 승객 데이터 셋을 고쳐보자
- 6.16 히트맵을 알아보자



이 장에서 배울 것들

- 데이터의 상관관계와 인과관계에 대하여 알아보시다.
- 고급 시각화 라이브러리인 시본 라이브러리의 특징에 대하여 살펴봅시다.
- 시본 라이브러리와 맷플롯립의 특징과 차이점을 알아보시다.
- 상관 계수의 시각화 방법을 알아보시다.
- 시본의 tips 데이터, flight 데이터를 통해 고급 시각화 기법을 익혀봅시다.
- 다차원 데이터를 시각화하는 도구인 파셋그리드에 대하여 살펴봅시다.
- 쌍그래프와 선형 회귀에 대해서 간략하게 살펴봅시다.





6.1 데이터 사이의 관련성을 알아보자



- 데이터 분석을 위해서는 측정가능한 형태로 된 자료를 모으고, 분석하고, 결과를 제시하는 것이 필요하며, 이러한 일은 **통계학**이라고 하는 학문의 주요 주제
- 통계에서는 측정한 결과와 조사 대상에 따라 다른 값으로 나타날 수 있는 특성 혹은 속성을 **변수**variable라고 함
- 예를 들면 키, 몸무게, 연간 소득과 같은 속성이 **변수**가 될 수 있으며, 이와 같이 두 개의 변수들이 함께 변화하는 관계를 나타내는 통계적 척도를 **상관관계**correlation라고 함
- 또한, 이 변수들 사이의 상관관계에 대한 정도를 -1에서 1 사이의 수치값으로 나타낸 것을 **상관계수**correlation coefficient라고 함



6.1 데이터 사이의 관련성을 알아보자



- 상관관계가 있는 두 변수가 있을 경우 한 값이 증가할 때 다른 값도 증가할 경우 양의 상관관계가 있다고 하며, 반대의 경우 음의 상관관계가 있다고 함

- 양의 상관관계와 음의 상관관계에 대한 예

- 양의 상관관계: 일평균 기온이 높으면(+) 아이스크림 판매량이 증가한다(+).
- 음의 상관관계: 일평균 기온이 높으면(+) 패딩 의류 판매량이 감소한다(-).

- 상관관계와 **인과관계**causation는 착각을 많이 일으키는 개념인데 예를 들어 다음과 같은 주장에 대해 생각해 보도록 하자.

아이스크림 판매량과 익사 사고건수는 상관관계가 매우 높다. 따라서 익사 사고를 예방하기 위해서 아이스크림 판매를 제한해야 한다.



6.1 데이터 사이의 관련성을 알아보자



- 일반적으로 아이스크림 판매량은 평균 기온이 높아지면 증가하므로, 일평균 기온은 아이스크림의 판매에 영향을 미치는 인과관계가 있다고 생각할 수 있음
- 하지만 익사 사고 역시 일평균 기온의 상승에 따른 야외 물놀이 활동의 증가로 인해 발생하는 사고로 볼 수 있음
- 따라서 아이스크림의 판매량과 익사 사고는 높은 상관도는 있을 수 있으나 인과성을 찾기는 어려움

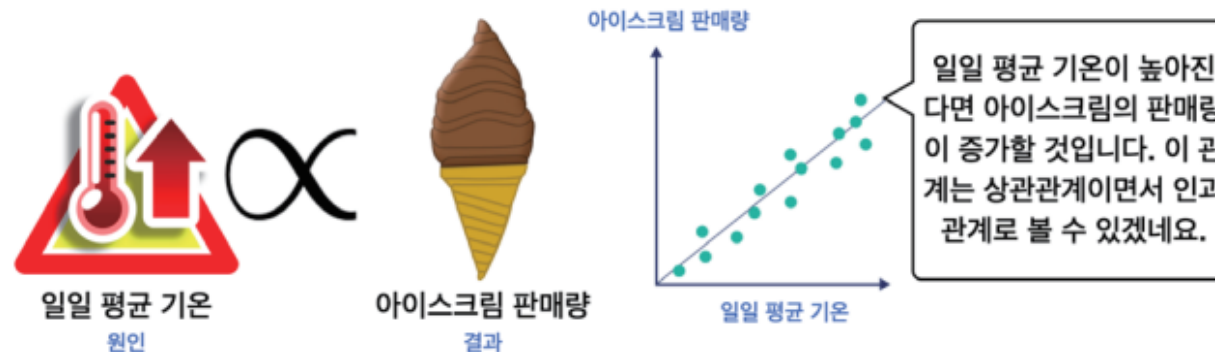




6.1 데이터 사이의 관련성을 알아보자



- 이와 같이 데이터를 가지고 어떠한 상관관계에 대해 주장을 하는 경우, 항상 **“이것이 인과관계를 보장해 주는가?”**라는 의문을 가지는 것이 필요
- **한 변수가 다른 변수의 직접적인 원인이 되는 관계가 인과관계**인데, 인과관계는 일반적으로 시간적 선후 관계를 포함함 (원인이 결과보다 먼저 발생)
- 인과관계가 아니면서도 두 변수가 서로 영향을 주는 관계 중에서 대표적인 것이 상관관계
- 즉, 상관관계를 가지는 두 변수 A, B가 있을 경우 **“A가 증가하면 B도 증가한다”**는 관계를 설명할 수 있지만, A가 B를 유발하거나 반대로 B가 A를 유발한다고 말할 수는 없음

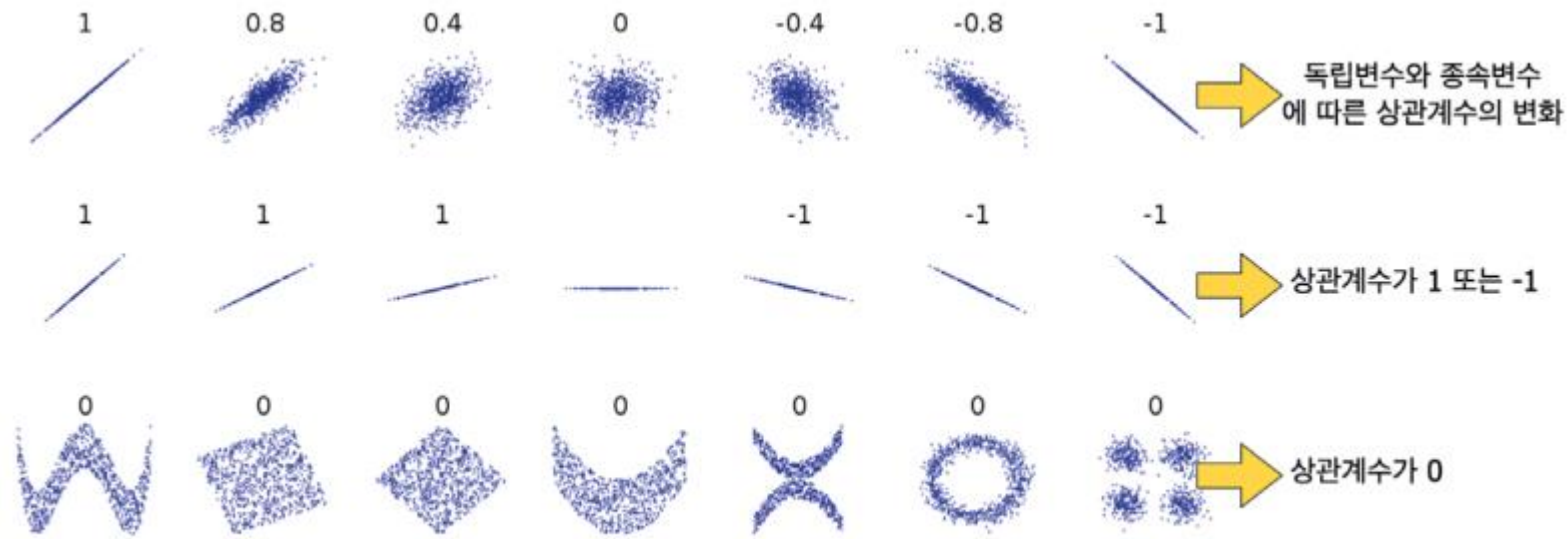




6.1 데이터 사이의 관련성을 알아보자



- 독립변수를 이차원 평면의 x, y 축에 각각 두고 데이터의 분포를 시각화한 그림 (출처: 위키피디아)



다양한 데이터의 집합과 상관관계를 보여주는 상관계수값

- 상관계수는 -1에서 1 사이의 값을 가지며 1일 경우 완전한 양의 상관관계, -1일 경우 완전한 음의 상관관계를 가진다고 할 수 있음
- 상관계수가 0에 가까울수록 상관관계가 약하다고 할 수 있음



6.1 데이터 사이의 관련성을 알아보자



- 특정 지역의 주택 면적과 최근 2년의 거래 가격의 관계를 조사하는 경우를 생각해 보면, 일반적으로 주택의 면적이 큰 경우 거래 가격도 높은 경우가 많을 것임
- 여기서 다른 변수에 영향을 덜 받는 변수인 **주택의 면적**은 **독립변수**가 되며, 이에 영향을 받아서 변화할 수 있는 **거래 가격**이 **종속변수**가 됨
- 종속변수는 연구자가 **독립변수의 변화에 따라 어떻게 변하는가 알고 싶은 변수**를 말함
- 이제 앞서 소개한 그림을 다시 살펴보면 다음과 같은 해석이 가능함



6.1 데이터 사이의 관련성을 알아보자

- 1) 그림의 가장 위쪽 줄에 있는 분포를 살펴보자. 이 분포는 **독립변수와 종속변수에 따른 상관계수의** 변화를 보여주고 있다. 둘 사이의 연관성이 클 경우 데이터는 직선의 주위에 분포한다. 이 연관성이 가장 클 때 상관계수는 **1**이 된다. 반면 둘 사이의 관계가 밀접하지 않을 경우, 즉 연관성이 약한 경우 데이터는 임의의 분포를 가지며 상관계수 값은 **0**이 된다.
- 2) 그림의 가운데 줄 분포를 보면 직선의 기울기에 관계없이 데이터가 직선의 주위에 몰려있을 경우 그 값은 **1**이 됨을 알 수 있다. 그림의 오른쪽은 기울기가 음의 값을 가지는 데이터인데 이 경우 상관계수는 **-1**이 된다.
- 3) 아래쪽 그림을 살펴보면 어떤 그림은 일정한 관계가 있다고 볼 수 있지만 실제 상관도는 **0**으로 나타난다. 이것은 상관계수가 **두 변수들 사이의 선형적인 관계만을 측정하는 한계**가 있기 때문이다.



6.1 데이터 사이의 관련성을 알아보자



- 4장에서 배운 넘파이의 `corrcoef()` 함수는 수학적으로 **피어슨** Pearson 상관계수를 계산하는 함수로, `corrcoef(x, y)`와 같이 `x, y`를 입력값으로 넣어서 `x, y` 요소들의 상관관계를 계산함
- 피어슨 상관계수는 변수들 간의 관련성을 구하는 이변량 상관분석에서 가장 보편적으로 이용되는 방법
- `corrcoef()` 함수에 대한 자세한 설명은 4장 10절의 내용을 참고



6.2 상관계수를 넘파이로 표현하기



- 0에서 9까지의 정수가 있는 다차원 배열 x 와, x 원소 값의 두 배 값을 가지는 다차원배열 $y1$ 사이의 상관관계를 계산해 보자.
- $y = 2x$ 로 나타낼 수 있는 이 식에서 두 변수는 선형적인 상관관계를 가지므로 상관계수는 1이 됨
- 따라서 `corrcoef()` 함수의 모든 성분 값은 1



```
import numpy as np      # 반복되는 내용으로 앞으로 삭제함

x = np.arange(0, 10)    # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9를 생성
y1 = x * 2              # 0, 2, 4, 6, 8, 10, 12, 14, 16, 18
np.corrcoef(x, y1)

array([[1., 1.],
       [1., 1.]])
```



6.2 상관계수를 넘파이로 표현하기



- 다음과 같이 계수를 0.2로 수정하더라도 선형적인 상관관계는 그대로이므로 **계수의 값과 무관하게 상관계수는 1**이 됨



```
x = np.arange(0, 10)      # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9를 생성
y2 = x * .2               # 0, .2, .4, .6, .8, 1.0, 1.2, 1.4, 1.6, 1.8
np.corrcoef(x, y2)

array([[1., 1.],
       [1., 1.]])
```



6.2 상관계수를 넘파이로 표현하기



- 이제 다음과 같이 x 의 세제곱 x^3 을 원소로 가지는 $y3$ 을 만들고 x 와 $y3$ 의 상관관계를 살펴보자.



```
x = np.arange(0, 10)    # 0, 1, 2, 3, 4, 5, 6, 7, 8, 9를 생성
y3 = x ** 3             # 0, 1, 8, 27,.. 과 같은 x의 세제곱 값을 원소로 함
np.corrcoef(x, y3)
```

```
array([[1.          , 0.90843373],
       [0.90843373, 1.          ]])
```

- 이 경우 x 의 값이 증가하면 $y3$ 의 값도 증가하는 관계에 있지만 선형적인 관계는 아님
- 하지만 두 변수는 여전히 동시에 증가하는 특징이 있으므로 0.9 정도의 높은 상관계수를 보임



6.2 상관계수를 넘파일로 표현하기



- 다음으로 0에서 100 사이의 난수를 10개 생성한 후 y4에 넣어 x와의 상관관계를 살펴보자.



```
np.random.seed(84) # 이 책의 결과값과 일치시키기 위한 시드 값
x = np.arange(0, 10) # 0에서 9 사이의 연속적인 수를 생성
y4 = np.random.randint(0, 100, size=10) # 0에서 100 사이 10개의 난수 생성
np.corrcoef(x, y4)
```

```
array([[1.          , 0.06818812],
       [0.06818812, 1.          ]])
```

- 난수란 특정한 규칙이 없는 수이므로 상관계수는 0에 가까운 값으로 나타나는 것을 알 수 있음



6.2 상관계수를 넘파일로 표현하기



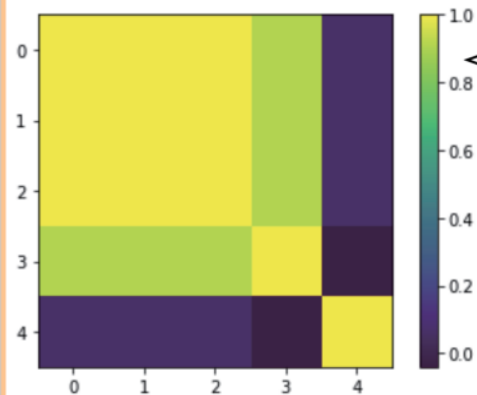
- 이제 x, y_1, y_2, y_3, y_4 다섯 개의 변수의 상관계수를 출력해 본 후 맷플롯립을 이용하여 시각화하자.

```
▶ result = np.corrcoef((x, y1, y2, y3, y4)) # 튜플 형태의 입력을 사용
print(result)
```

```
[[ 1.          1.          1.          0.90843373  0.06818812]
 [ 1.          1.          1.          0.90843373  0.06818812]
 [ 1.          1.          1.          0.90843373  0.06818812]
 [ 0.90843373  0.90843373  0.90843373  1.          -0.04068803]
 [ 0.06818812  0.06818812  0.06818812 -0.04068803  1.          ]]
```

```
▶ import matplotlib.pyplot as plt

plt.imshow(result)
plt.colorbar()
```



상관계수 행렬을 색상으로 시각화해 봅시다. 결과와 같이 상관계수가 클수록 노란색으로 나타납니다. 반면 상관계수가 0에 가까울 어두운 청색으로 나타나는 것을 볼 수 있네요.



6.2 상관계수를 넘파일로 표현하기



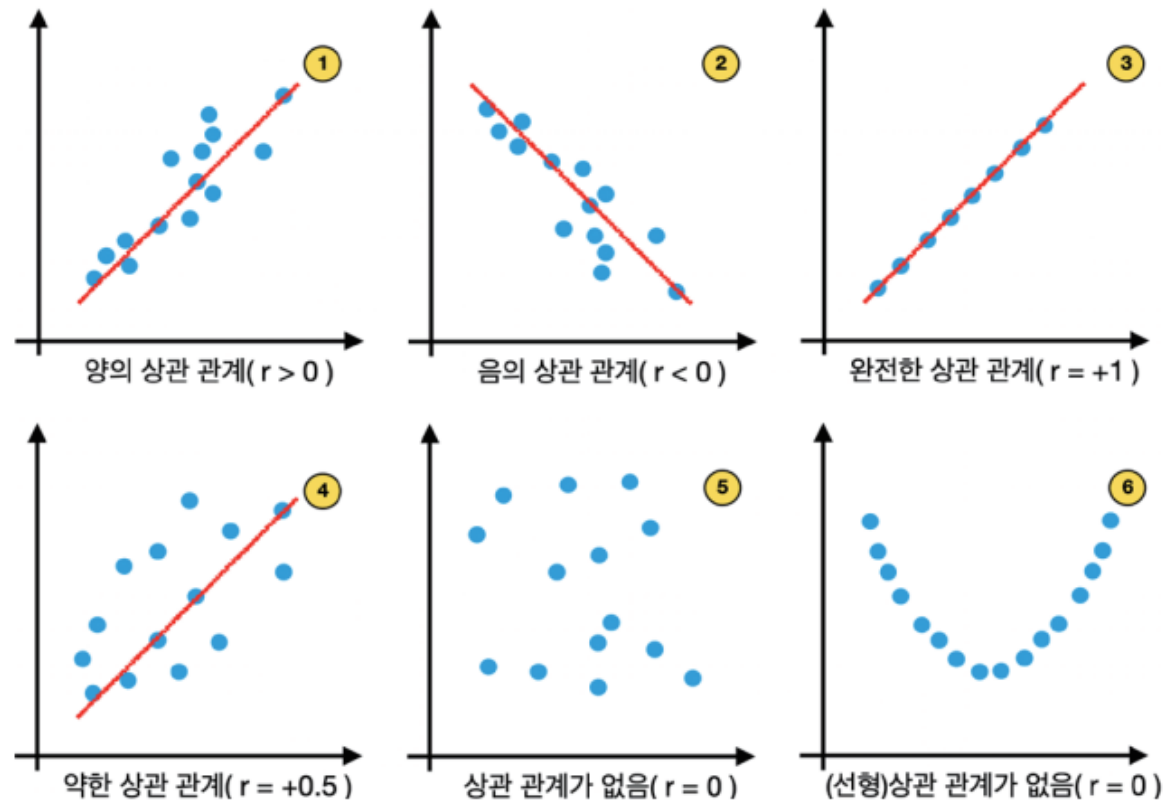
- 상관계수 값을 가지는 배열인 `result`를 위와 같이 `imshow()` 함수로 시각화할 수 있음
- 대각 성분은 1이므로 노란색으로 밝게 나타나며, 대각 성분을 제외한 세로, 가로 성분에서 가장 어둡게 나타나는 부분은 상관계수가 0에 가까운 값을 가지는 경우임
- 이러한 형태의 표현 방법을 **히트맵** `heatmap`이라고 함



6.3 상관계수의 시각화와 해석



- 앞서 살펴본 상관계수를 r 이라고 하고 키와 몸무게와 같은 독립 변수를 각각 x, y 축에 배치한 후 데이터를 시각화한 그림을 살펴보자.
- 그림에서 파란색 점은 데이터의 분포를 나타내며, 빨간색 실선은 이 데이터의 분포를 가장 잘 설명하는 직선이다.





6.3 상관계수의 시각화와 해석



- ① r 의 부호가 +인 경우: 두 변수 x , y 의 값 중에서 한쪽이 증가할 때 한쪽이 증가하는 관계가 있음.
- ② r 의 부호가 -인 경우: 두 변수 x , y 의 값 중에서 한쪽이 증가할 때 한쪽은 감소하는 관계가 있음.
- ③ $r = +1$ 인 경우: x , y 가 서로 완벽한 선형 함수로 모델링이 가능할 경우 선형 함수의 기울기 값에 상관없이 상관계수는 1이 됨.
- ④ $r = +0.5$ 인 경우: r 값은 분산이 커질수록 작아짐.
- ⑤ $r = 0$ 인 경우: 데이터의 분포가 랜덤할 경우 상관관계가 없으므로 상관계수는 0이 됨.
- ⑥ $r = 0$ 인 경우: 상관계수는 선형적인 상관도만을 측정하므로 $y = x^2$ 와 같은 비선형적 관계일 경우 0에 가까운 값을 가짐(즉, 선형 상관관계가 없음).



6.3 상관계수의 시각화와 해석



6.4 간단한 시본 튜토리얼로 시작하기



- **시본 라이브러리**는 맷플롯립을 기반으로 만들어졌으며, 맷플롯립에 비하여 높은 수준의 인터페이스를 제공하여 사용자들이 보다 쉽게 데이터를 분석하고 시각화할 수 있음
- 시본 라이브러리의 주요 특징들
 - 맷플롯립 라이브러리를 기반으로 하고 있으며 세련된 그림 라이브러리를 제공하고 있어서, 고급스러운 그림을 쉽게 그릴 수 있다.
 - 정교한 그래픽 제어를 할 때는 맷플롯립과 함께 사용할 수 있다.
 - 여러 종류의 풍부한 데이터 집합을 제공하고 있어서 데이터 분석에 좀 더 적합하다.



6.4 간단한 시본 튜토리얼로 시작하기



- 시본에서 제공하는 데이터 집합 중에서 **가장 기본적인 데이터 중의 하나인 tips 데이터 셋을 사용하여 시본을 익혀보자.**



```
import seaborn as sns
```

```
# 테마를 설정하는 기능이 있음
```

```
sns.set_theme(style="darkgrid")
```

```
# 시본에서 제공하는 팁 데이터를 가져오자
```

```
tips = sns.load_dataset("tips")
```

```
# 팁 데이터의 시각화 기능을 호출하자
```

```
sns.relplot(data=tips, x="total_bill", y="tip", col="time",\n            hue="smoker", style="smoker", size="size")
```

- numpy를 나타내는 별칭으로 np, matplotlib을 mpl이라는 별칭으로 사용하듯 seaborn은 sns라는 별칭을 사용함

```
import seaborn as sns
```



6.4 간단한 시본 튜토리얼로 시작하기



- 시본 라이브러리는 다양한 테마를 제공하고 있으며 사용자들의 필요에 따라서 그래픽 이미지의 표현을 변경할 수 있는데, 이를 위해서 사용하는 함수가 `set_theme()` 함수

```
sns.set_theme(style="darkgrid")
```

- `load_dataset()` 함수는 미리 만들어진 데이터 집합을 불러오는 함수이며, 데이터 셋 객체를 반환함
- `tips`는 식당의 손님들이 식사를 하고 얼마만큼의 팁을 지불하였는지를 조사하여 만든 데이터 집합

```
tips = sns.load_dataset("tips")
```





6.4 간단한 시본 튜토리얼로 시작하기

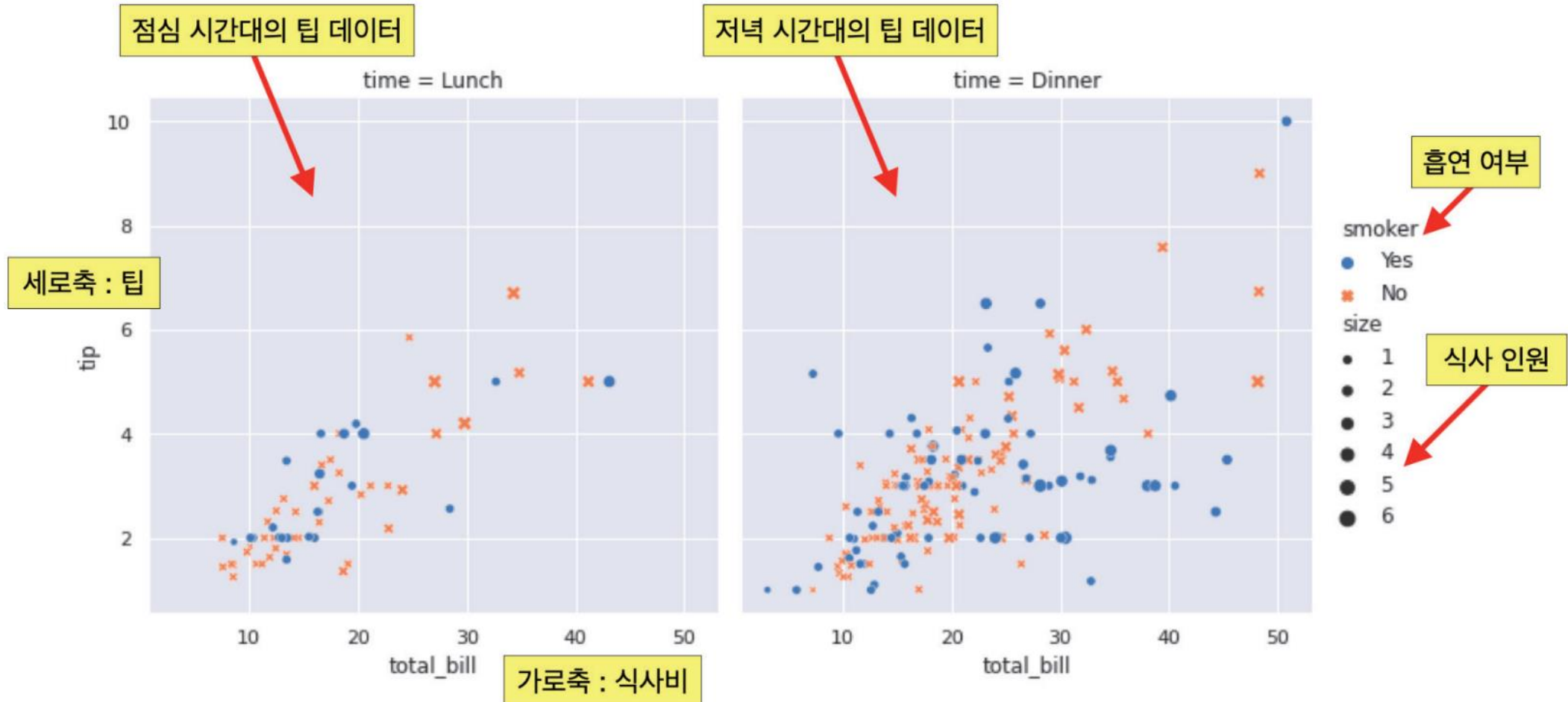


- relplot() 함수는 주어진 데이터를 이용하여 산점도 그래프와 선 그래프를 모두 그릴 수 있는 함수
- 데이터의 x축 값과 y축 값을 각각 'total_bill'과 'tip'으로 주어 손님들의 식사비와 팁의 상관관계를 살펴보는 것이 목표
- 이때, col 키워드 인자의 값은 'time'으로 지정하여 점심(Lunch)과 저녁(Dinner) 시간대의 팁을 따로 분석

```
sns.relplot(data=tips, x="total_bill", y="tip", col="time",\n            hue="smoker", style="smoker", size="size")
```



6.4 간단한 시본 튜토리얼로 시작하기



tips 데이터를 시본 라이브러리로 시각화한 결과



6.5 tips 데이터의 구조



- `load_dataset('tips')` 메소드가 반환하는 것은 판다스의 **데이터프레임** 객체이며 다음과 같이 `head()` 메소드를 사용해서 그 내부의 값을 살펴볼 수 있음

 `tips.head()` # tips 데이터의 데이터프레임 살펴보기

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

- 데이터프레임은 위의 결과와 같이 행과 열로 이루어진 이차원 데이터를 말함



6.5 tips 데이터의 구조



- 이 데이터프레임의 각 열은 7개의 특성으로 이루어져 있으며, 각 특성의 의미는 다음과 같음

특성	해석	특성	해석
total_bill	계산서의 식사 요금	day	요일(Thur, Fri, Sat, Sun만 제공됨)
tip	팁	time	점심(Lunch)/저녁(Dinner) 식사의 구분
sex	여성(Female)/남성(Male)	size	식사 인원
smoker	흡연(Yes)/비흡연(No)		

- 전체 데이터의 수를 shape 속성으로 살펴보면 244개의 데이터 셋이 7개의 속성을 가지는 것을 확인할 수 있음

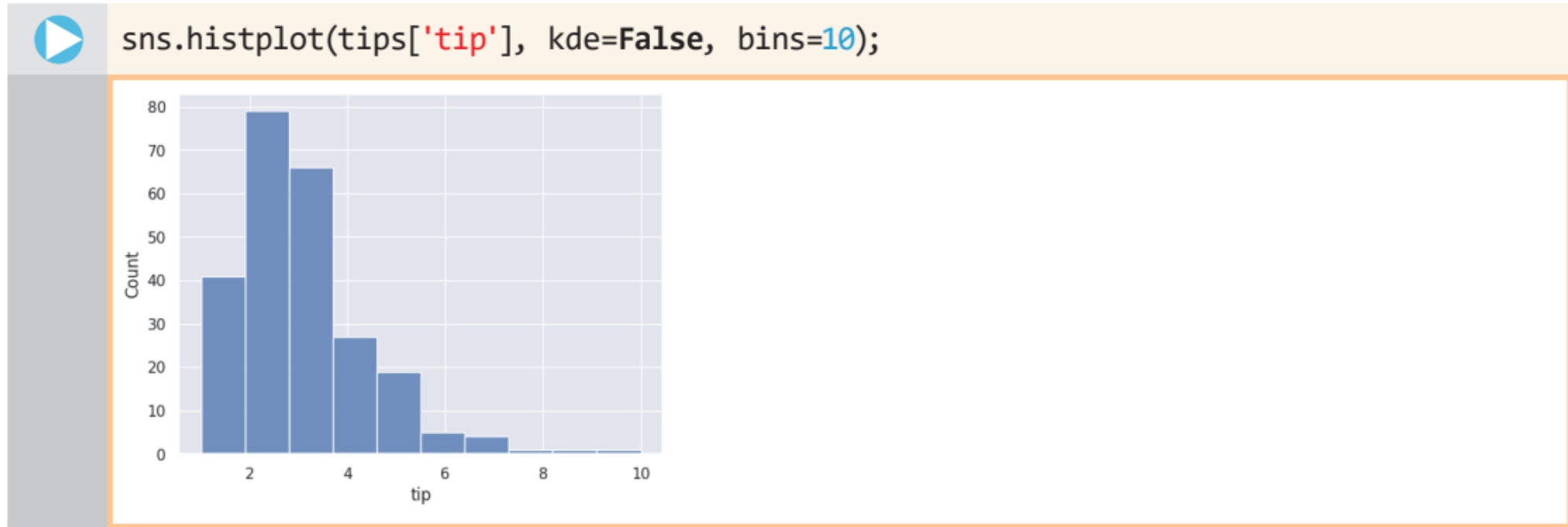
```
tips.shape
(244, 7)
```



6.5 tips 데이터의 구조



- 만약 고객의 팁 액수에만 관심을 가진다면, 히스토그램을 사용하여 팁의 분포를 살펴볼 수 있음
- 판다스 데이터프레임에서 특정한 열을 추출하기 위해서는 [] 내에 열의 이름(특성)을 입력



- 시본의 histplot()은 데이터의 분포를 히스토그램으로 나타내는 함수



6.5 tips 데이터의 구조



- 함수의 키워드 인자 중에서 bins를 12로 늘리고 결과를 살펴보면 그림과 같이 가로축이 더욱 촘촘하게 분할된 것을 볼 수 있음



- 이를 통해 고객들이 1달러에서 4달러가량의 돈을 팁으로 많이 지출하는 것을 알 수 있음



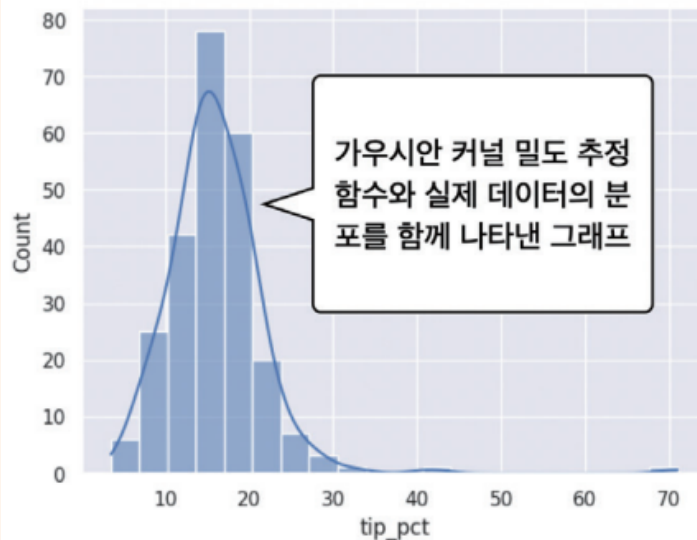
6.5 tips 데이터의 구조



- 보다 더 의미있는 정보를 얻기 위해 실제 식사 요금 대비 팁을 히스토그램으로 나타내어 보자.
- 다음과 같은 식을 통해 'tip_pct'라는 새로운 필드를 만든 후에 이 필드의 값을 히스토그램으로 살펴보자.



```
tips['tip_pct'] = 100 * tips['tip'] / tips['total_bill']  
sns.histplot(tips['tip_pct'], kde=True, bins=20)
```

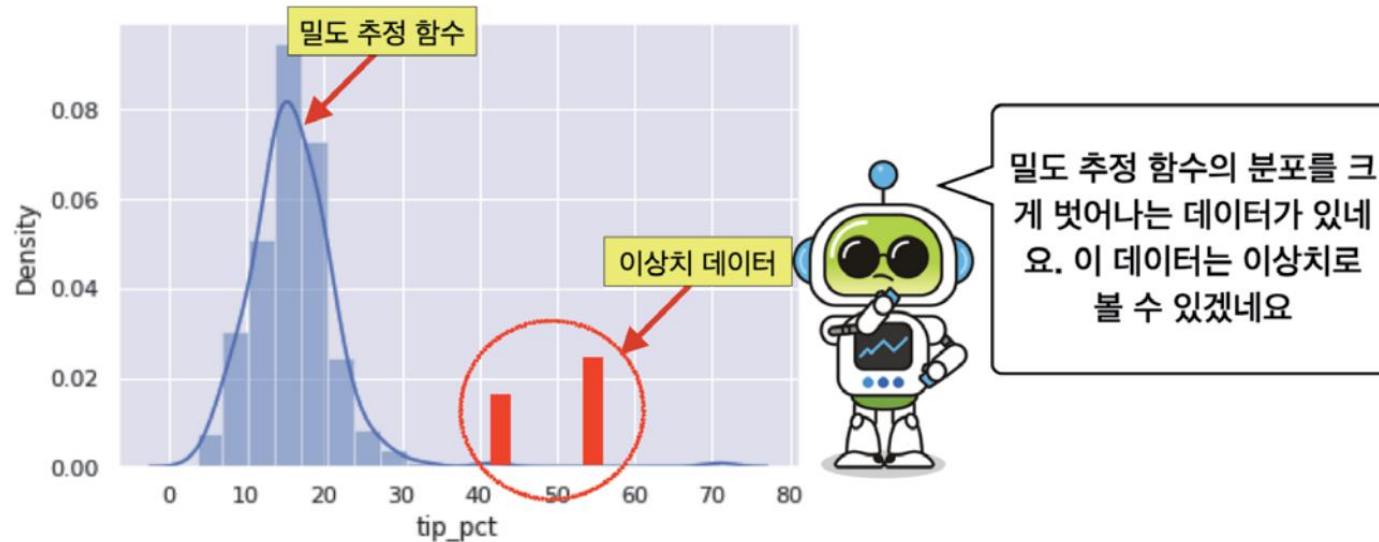




6.5 tips 데이터의 구조



- 여기서 사용된 `kde=True` 키워드 인자는 **가우시안 커널 밀도 추정** (gaussian kernel density estimation) 을 실선으로 나타내어 주는 역할을 함



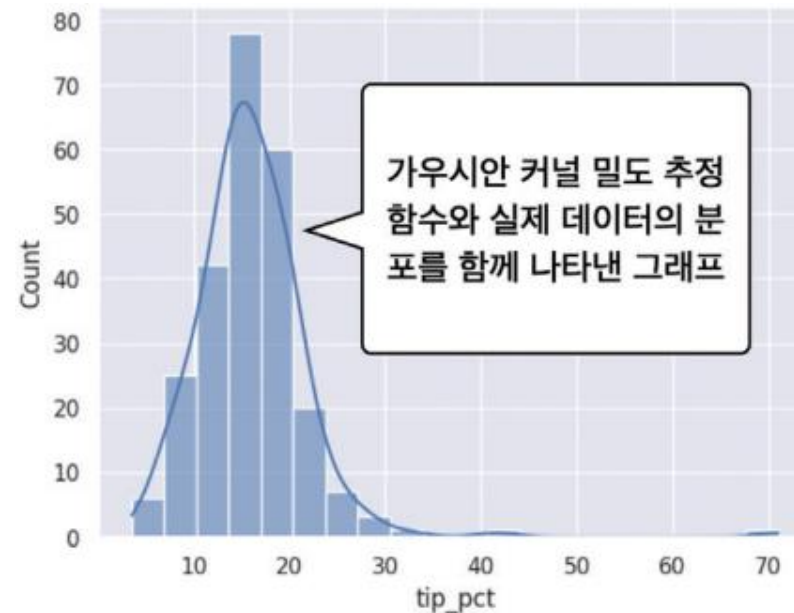
- 이상치 탐지**란 정상 데이터 또는 일반적인 데이터에 비하여 그 분포가 현저하게 차이가 나는 데이터를 탐지하는 기법
- 밀도 기반 이상치 탐지법**이란 데이터의 분포를 사용하여 이상치를 찾아내는 방법



6.5 tips 데이터의 구조



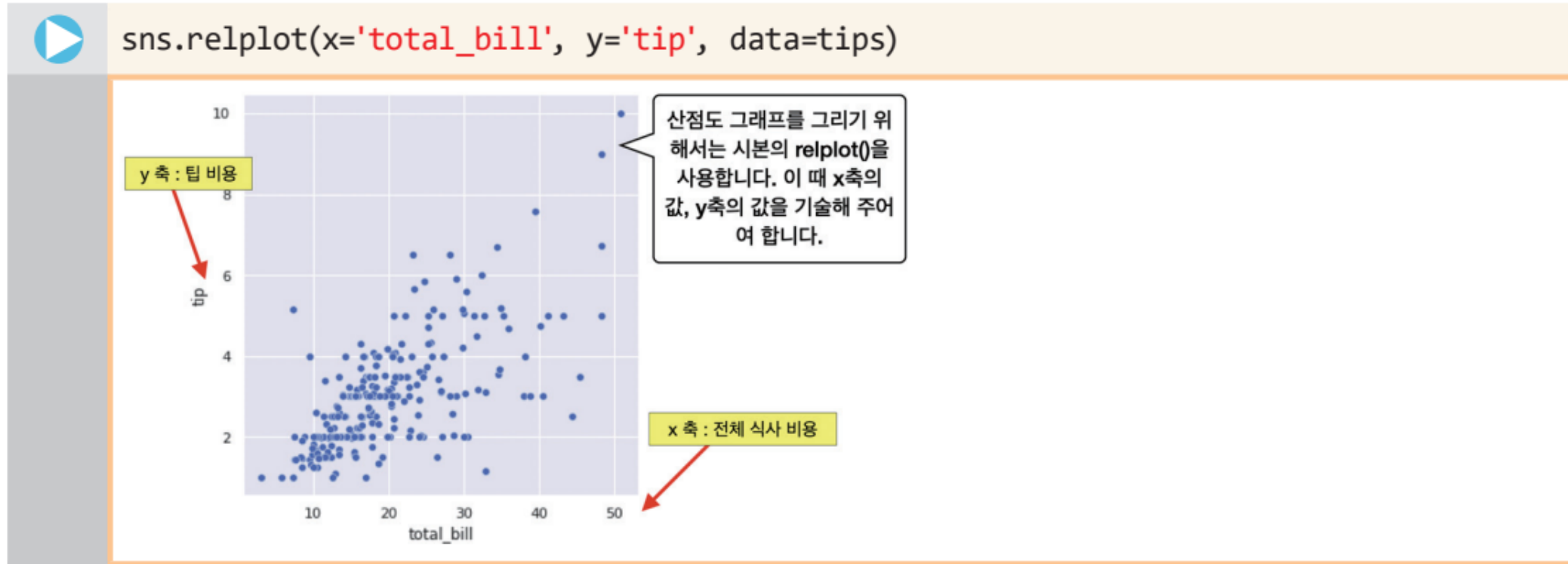
- 이 방법은 기존 데이터의 분포를 사용하여 데이터에 맞는 분포를 추정하고, 이렇게 추정된 분포를 이용하여 기존 데이터 중에서 이상치를 찾아내거나, 새로운 데이터가 들어왔을 때 이상치인지 확인함
- 가우시안 커널 밀도 추정을 통해서 그림과 같이 실제 데이터의 분포와 밀도 추정 함수가 추정한 분포를 비교해 볼 수 있음
- 이를 통해 팁 퍼센트는 15%가량의 구간에서 가장 많이 분포하는 것을 볼 수 있으며, **정규 분포 함수와 유사한 패턴**을 가지는 것도 알 수 있음





6.6 산점도 그래프로 관계를 상세하게 나타내보자

- 이어서 다음과 같은 방법으로 산점도 그래프를 그려보도록 하자.



- 산점도 그래프를 통해서 식사 요금과 팁과의 상관관계를 확인해 볼 수 있음



6.6 산점도 그래프로 관계를 상세하게 나타내보자



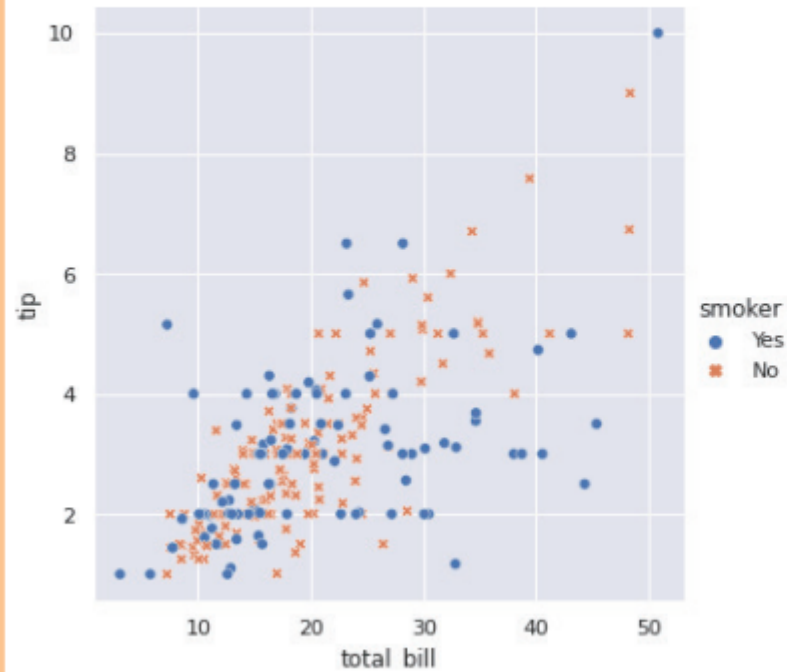
- seaborn의 relplot() 함수는 x='total_bill' 키워드 인자를 통해 x축의 값으로 전체 식사 비용을, y='tip' 키워드 인자를 통해 y축의 값으로 팁 비용을 그려줌
- 산점도 그림과 함께 hue, style이라는 키워드 인자를 통해서 그룹을 묶는 것도 가능
- hue 키워드의 인자 'smoker'는 smoker 열이 가질 수 있는 값 True, False를 서로 다른 색으로 표시하며, style은 이 열의 서로 다른 값에 대해서 스타일을 다르게 만들어 줌
- 따라서 다음과 같이 **청색 동그라미 스타일과 주황색 십자가 스타일**의 형태로 흡연 여부를 구분하여 표시해 줌



6.6 산점도 그래프로 관계를 상세하게 나타내보자



```
sns.relplot(x='total_bill', y='tip', hue='smoker',\n            style='smoker', data=tips)
```





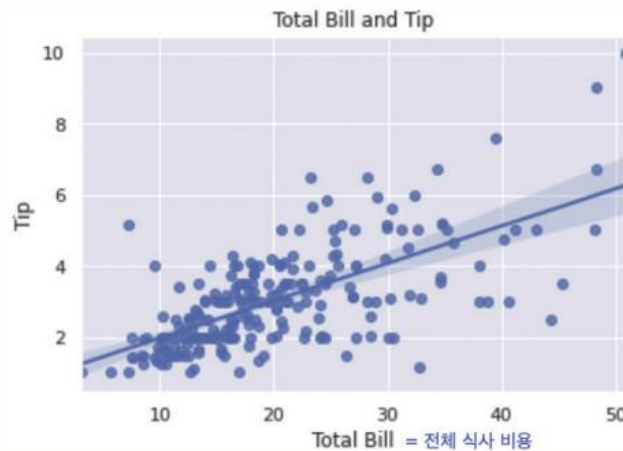
6.6 산점도 그래프로 관계를 상세하게 나타내보자



- 산점도 그래프를 보면 식사 요금과 팁 사이에는 선형적인 상관관계가 있는 것처럼 보이는데 이것을 직선을 그어 확인하는 방법으로 `regplot()` 함수가 있음
- 이 함수를 이용하여 `axes` 객체를 만들고 이 객체에 x축, y축 레이블, 그리고 제목을 넣을 수 있음
- 그림을 살펴보면 데이터의 분포를 잘 표현하는 직선이 나타나는 것을 볼 수 있는데 이 직선이 바로 **선형 회귀 직선**



```
ax = sns.regplot(data=tips, x='total_bill', y='tip')
ax.set_xlabel('Total Bill')           # x축의 레이블
ax.set_ylabel('Tip')                  # y축의 레이블
ax.set_title('Total Bill and Tip')    # 그림의 제목
```

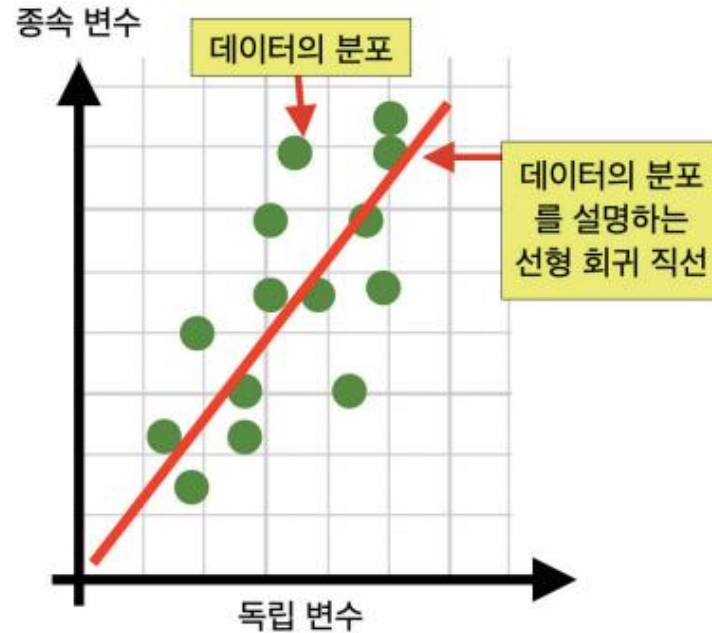


두 데이터의 상관관계를
가장 잘 표현하는 직선
= 선형 회귀 직선



6.6 산점도 그래프로 관계를 상세하게 나타내보자

- **선형 회귀**에 대한 개략적인 설명을 위해 그림과 같이 2차원 평면에 데이터가 분포한다고 가정하자.



- 이 평면의 x 축을 독립변수 값이라고 하고, y 축을 종속변수 값이라고 하면 이 데이터의 분포를 통해 독립변수와 종속변수가 어느 정도 상관성을 가지고 있다고 볼 수 있음
- 독립변수가 종속변수에 영향을 주기 때문에 이 데이터의 분포를 가장 잘 설명하는 선형함수가 만들어 질 수 있는데 이것을 **선형 회귀 직선**이라고 함



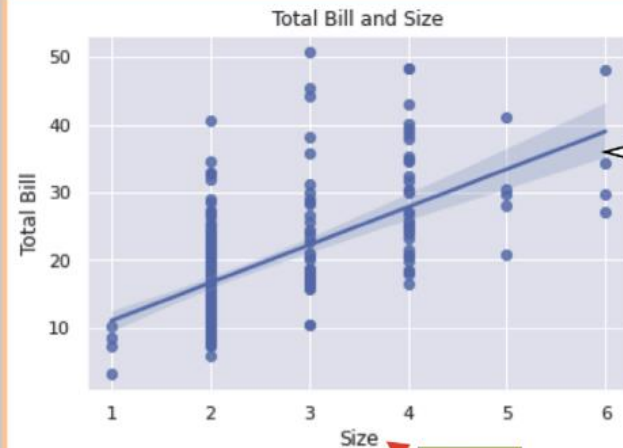
6.6 산점도 그래프로 관계를 상세하게 나타내보자



- 이제 식사 인원과 식사 요금 사이의 관계 파악을 위해 다음과 같이 `x = 'size', y = 'total_bill'`로 코드를 수정해 보자.



```
ax = sns.regplot(data=tips, x='size', y='total_bill')
ax.set_xlabel('Size')           # x 축의 레이블
ax.set_ylabel('Total Bill')     # y 축의 레이블
ax.set_title('Total Bill and Size') # 그림의 제목
```



식사 인원과 식사비 총액 사이의 선형적인 상관관계를 볼 수 있군요.

- 식사 인원과 식사 요금은 상관관계가 있으나 이 그래프의 결과는 이전의 결과와는 다소 다름
- 식사 인원은 1명, 2명, 4명과 같이 이산적이기 때문에 그 시각화 결과는 1, 2, 3, 4, 5, 6의 불연속적인 숫자 구간에서만 나타남



6.7 변수 사이의 관계를 알아보기에 편리한 쌍 그래프



- 쌍 그래프는 여러 변수들 사이의 관계를 한 눈에 살펴보는데 편리함
- 앞에서 살펴본 tips 데이터프레임을 쌍 그래프로 표시하기 위해서는 다음 방법을 사용

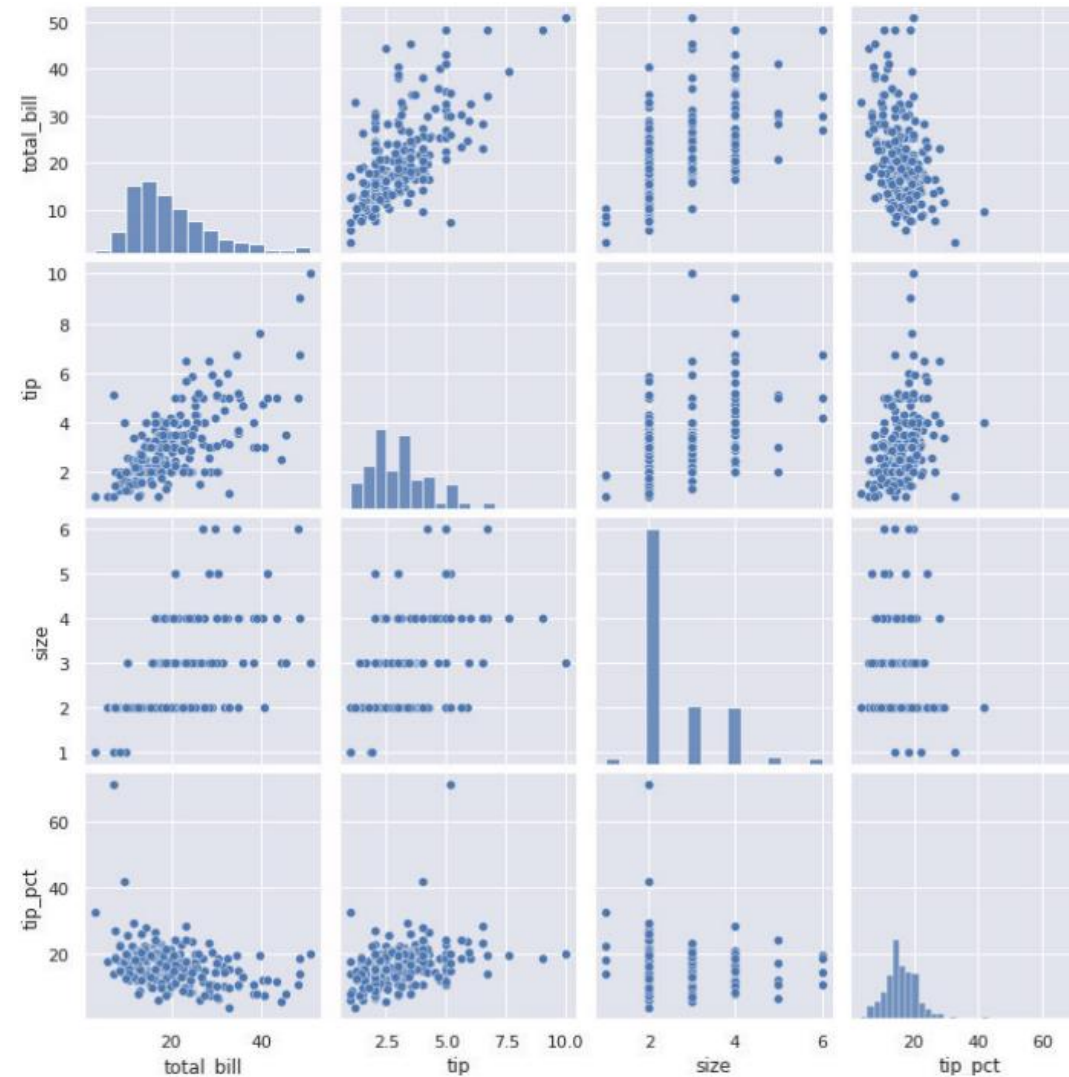


```
sns.pairplot(tips)
```

- 이 함수의 출력 결과는 다음과 같음
- 상관도의 모든 대각 성분은 항상 1이므로 그림을 그리는 것이 큰 의미 없기에 대각 성분은 각 변수 값의 분포를 나타내는 히스토그램으로 표현함



6.7 변수 사이의 관계를 알아보기에 편리한 쌍 그래프





6.7 변수 사이의 관계를 알아보기에 편리한 쌍 그래프



- 데이터의 분포와 그에 따른 해석

분포 그림			
분포 그림에 대한 해석	두 변수는 모두 연속적이며, 상관도가 높다.	두 변수는 상관도가 높으나 한 변수는 연속적이며 다른 변수는 이산적인 값이다.	두 변수는 모두 연속적이며 상관도가 비교적 낮다.

- 손님의 수에 해당하는 size 변수가 있는 행과 열은 모두 이산적인 값이 들어가기 때문에 불연속적으로 표시된다는 특징이 있음



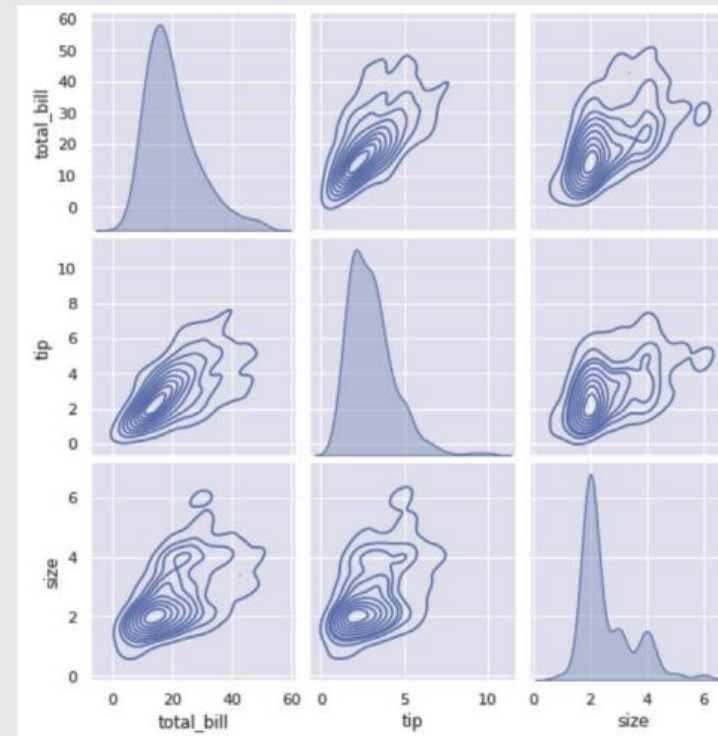
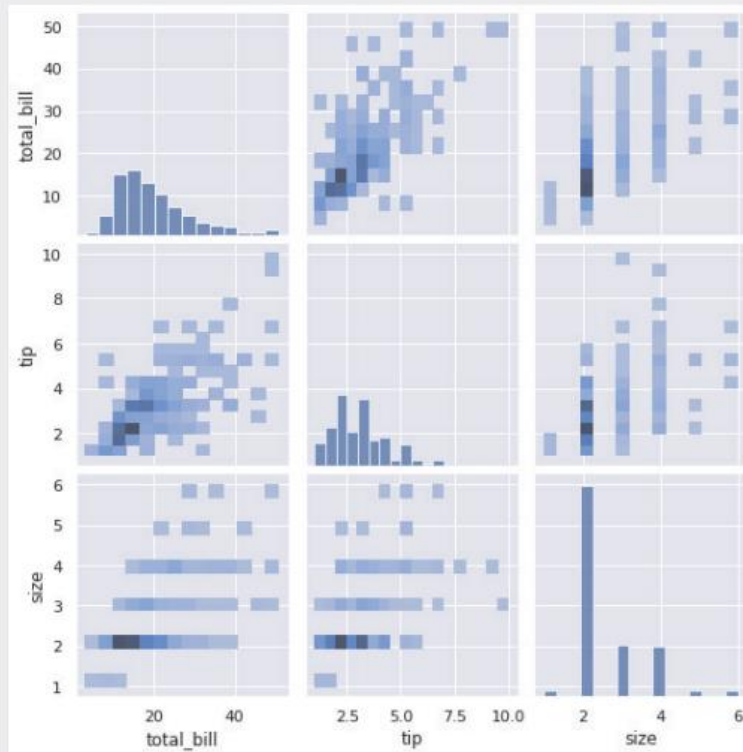
6.7 변수 사이의 관계를 알아보기에 편리한 쌍 그래프



도전문제 6.1 pairplot()의 kind 사용하기

난이도 중

- (1) 시본 라이브러리의 pairplot() 함수에 kind='hist' 키워드 인자를 추가하여 왼쪽 그림과 같이 표시하여라.
- (2) 다음으로 kind='kde'를 사용하여 오른쪽과 같이 쌍 플롯을 그려 보아라.





6.7 변수 사이의 관계를 알아보기에 편리한 쌍 그래프



NOTE 데이터 과학자들의 놀이터 캐글

오늘날 인터넷과 소셜 미디어, 사물인터넷으로부터 생성되는 많은 데이터는 이 책의 예제에서와 같이 단순하지 않으며 정형, 비정형의 다양한 형태를 가지고 있다. 캐글(kaggle.com)은 2010년에 설립된 데이터 예측 모델과 분석을 위한 플랫폼이다. 이 기업 및 단체에서는 자신들의 데이터와 함께 해결 과제를 등록할 수 있고, 데이터 과학자들은 이 데이터를 가지고 문제를 해결하기 위하여 서로 경쟁하고 협력하며 모델을 개발한다. 이러한 협업 플랫폼을 통해서 새로운 데이터 분석, 인공지능 모델이 나타나며 좋은 성능을 보여주는 기술은 곧바로 전 세계의 인공지능 개발자들에게 공유된다.



6.8 FacetGrid 알아보기

- 시본 라이브러리의 파셋그리드(FacetGrid)는 다차원 데이터를 시각화하는 강력한 도구
- 이 클래스는 데이터의 서브셋을 격자(grid) 형태로 구성하여, 각 서브셋에 대해 다양한 그래프를 그릴 수 있게 해줌
- 파셋그리드의 주요 기능들

- **다중 패널 설정이 가능함:** 파셋그리드는 다중 패널을 만들어 각 패널에 데이터의 서로 다른 부분 집합을 표시하는 기능을 지원한다. 이 패널은 맷플롯립의 서브플롯 기능과 유사하다.
- **조건부 관계 표시:** 하나 이상의 변수에 따라 데이터를 구분하는 기능을 제공한다. 이 때 각 서브플롯에 대해 동일한 그래프 유형을 생성하여 변수의 조건에 따라 데이터의 분포를 쉽게 비교할 수 있다.
- **유연한 설정 기능:** 각 서브플롯의 크기, 비율, 색상 팔레트, 제목 등을 사용자가 유연하게 설정할 수 있다.



6.8 FacetGrid 알아보기



- tips 데이터 셋에서 점심 식사 시간, 저녁 식사 시간에 대하여 흡연자와 비흡연자가 각각 어느 정도 팁을 지출하였는가에 대한 정보를 얻고자 하는 경우, 2x2 크기의 다중 패널이 필요함
- 이 4개의 패널에는 각각 점심 식사 시간의 흡연자, 저녁 식사 시간의 흡연자, 점심 식사 시간의 비흡연자, 저녁 식사 시간의 비흡연자의 팁 정보가 나타나게 됨
- 이를 지원하는 다음과 같은 FacetGrid 코드를 살펴보자.
- 아래의 코드는 FacetGrid의 인자로 tips 데이터, col="time", row="smoker"를 통해 행의 값으로 흡연 여부, 열의 값으로 시간대를 지정하였음



6.8 FacetGrid 알아보기



내장 데이터 셋 로드

```
tips = sns.load_dataset('tips')
```

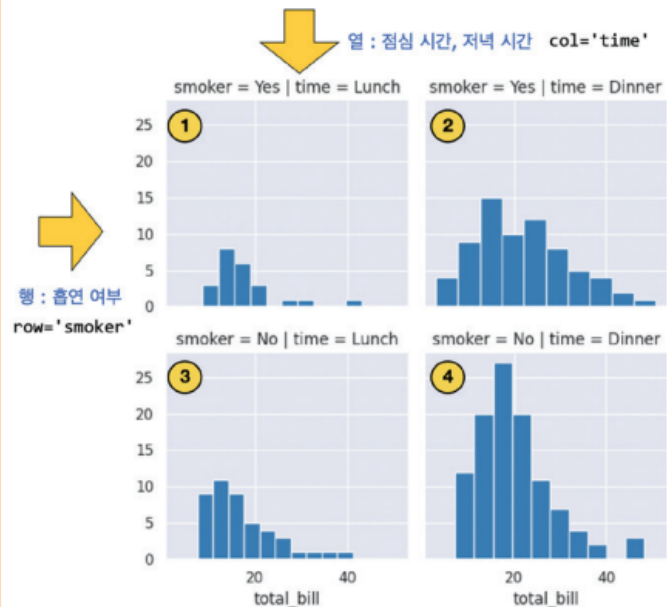
FacetGrid 객체 생성

'time'에 따라 다른 행을 생성하고, 'smoker'에 따라 다른 열을 생성

```
g = sns.FacetGrid(tips, row='smoker', col='time')
```

각 서브플롯에 히스토그램 그리기

```
g.map(plt.hist, 'total_bill')
```





6.8 FacetGrid 알아보기

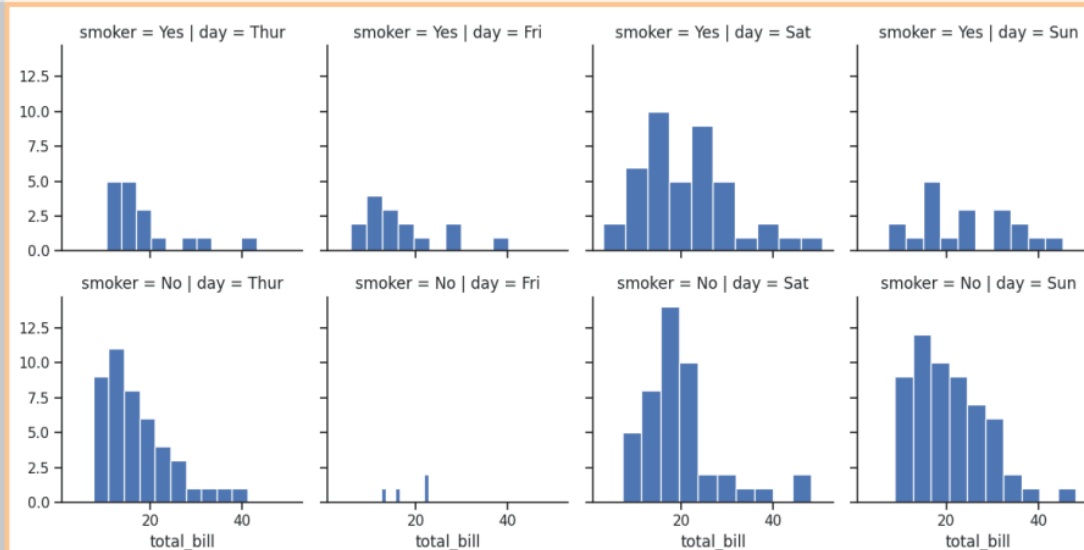


- 코드의 실행 결과에서 1, 2, 3, 4번 히스토그램은 각각 점심 식사 시간의 흡연자, 저녁 식사 시간의 흡연자, 점심 식사 시간의 비흡연자, 저녁 식사 시간의 비흡연자의 전체 식사 비용(total_bill) 정보를 담고 있음
- 아래 코드는 요일에 따른 흡연자와 비흡연자의 전체 식사 비용 지출에 대한 FacetGrid 시각화의 결과이다.



```
# 'day'에 따라 다른 행을 생성하고, 'smoker'에 따라 다른 열을 생성
g = sns.FacetGrid(tips, row='smoker', col='day')

# 각 서브플롯에 히스토그램 그리기
g.map(plt.hist, 'total_bill')
```





6.8 FacetGrid 알아보기

- 결과를 살펴보면 모두 2행 4열의 그리드가 생성된 것을 볼 수 있으며, 첫 번째 행은 흡연자, 두 번째 행은 비흡연자의 요일별 전체 식사비 지출액의 정보가 나타나 있음
- 파셋그리드의 `map()` 메소드의 첫 번째 인자에는 `plt.hist`와 같은 호출 가능한 플로팅 함수가 올 수 있음
- 두 번째 인자인 `'total_bill'`은 플로팅 함수가 그려야 할 데이터의 이름이며, 만일 `'total_bill'` 대신 `'tips'`가 온다면 전체 식사비가 아닌 지출한 팁이 나타나게 됨



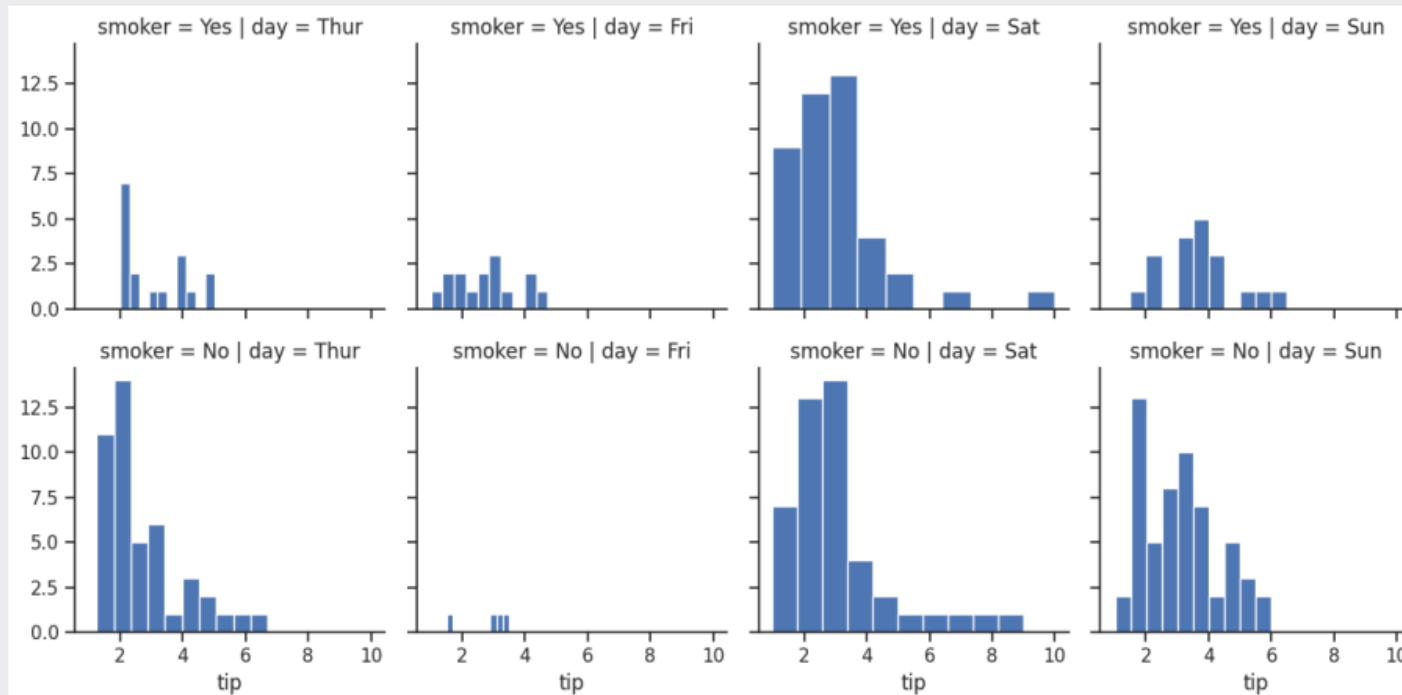
6.8 FacetGrid 알아보기



도전문제 6.2 FacetGrid를 사용하여 요일별 팁 지출 상황 시각화하기

난이도 하

이 절의 코드를 수정하여 요일별/흡연,비흡연자 별 팁 지출 상황을 다음과 같이 시각화하도록 하자. 행은 흡연 여부를 나타내며, 열은 목요일, 금요일, 토요일, 일요일을 나타내도록 하자(힌트: 위의 코드에서 'total_bill' 대신 'tip'을 사용하자).





LAB⁶⁻¹ groupby()를 이용한 그룹핑과 시각화



실습 시간



위의 코드에서 목요일, 금요일, 토요일, 일요일이라는 요일에 따른 데이터의 통계값을 보기 위해서는 `tips.groupby('day')`를 사용한다. `groupby()`는 판다스데이터 프레임의 강력한 그룹핑 기능이다. `groupby()` 기능을 사용하여 다음과 같은 데이터프레임을 완성하여라. 이 데이터프레임의 행은 Thur, Fri, Sat, Sun과 같은 요일이며, 열은 `total_bill`, `tip`, `size`와 같은 수치형 자료이다. 이 셀에 들어갈 값은 17.68, 2.77과 같은 평균 지출 비용이 되도록 하여라.

원하는 결과

	total_bill	tip	size
day			
Thur	17.682742	2.771452	2.451613
Fri	17.151579	2.734737	2.105263
Sat	20.441379	2.993103	2.517241
Sun	21.410000	3.255132	2.842105

힌트



`groupby()`를 이용하여 결과와 같이 요일 단위로 데이터를 묶을 경우 `tips.groupby('day')`를 사용한다.



LAB⁶⁻¹ groupby()를 이용한 그루핑과 시각화



해답 코드



```
tips = sns.load_dataset('tips')

df = tips.groupby('day', observed=True).mean(numeric_only=True)
df
```



NOTE 배열의 차원과 벡터의 차원

차원dimension은 어떤 수치 데이터를 다룰 때에 고려해야 하는 속성이 몇 개인지에 따라 결정된다. 예를 들어 3차원 공간의 위치는 x축 위의 위치, y축 위의 위치, 그리고 z축 위의 위치를 모두 고려해야만 정확한 한 점을 가리킬 수 있기 때문에 “3차원” 좌표로 표현한다.

차원이라는 용어를 많이 사용하는 데이터 구조에는 배열이 있다. 가장 단순한 배열은 데이터가 하나의 줄로 나열되어 있는 것이다. 이것을 1차원 배열이라고 부른다. 수학과 과학 분야에서 이런 수치 데이터를 **벡터vector**라고 부른다.

배열의 차원을 한 단계 높이면 1차원 배열을 여러 줄로 겹쳐 놓은 형태가 된다. 이때 어떤 항목을 지목하는 인덱스를 표현하려면 두 가지 방향으로 각각 하나씩의 위치값이 필요하고, 이 방향을 각각 배열의 축이라 부른다. 배열의 차원은 축의 개수이므로, 이런 배열은 2차원 배열이라 부른다. 수학에서는 각각의 축을 **행row**과 **열column**이라고 부르며, 이런 모양의 데이터를 **행렬matrix**라고 부른다.

같은 방식으로 배열의 차원을 높여 보자. 3차원 배열은 입체적인 모양이 될 것이다. 이런 구조가 표현하는 데이터는 벡터나 행렬이 아니라 **텐서tensor**라고 부른다. 텐서는 3차원 배열뿐만 아니라 모든 차원의 배열을 포괄하는 개념이므로 벡터는 1차원 텐서, 행렬은 2차원 텐서라 할 수 있다.

시각적으로 나타내기에는 다소 어렵지만 코드로는 4차원, 5차원 배열을 쉽게 만들 수 있다. 3차원 배열을 나열한 배열, 그렇게 만든 4차원 배열을 또 나열한 배열이다. 그리고 이 모든 것이 **텐서**이다.



LAB⁶⁻² 배열의 형태를 알아내고 슬라이싱하여 연산하기



실습 시간



어떤 병원의 환자들에 대한 임상 데이터를 얻고자 한다. 우선 다음과 같은 4명의 환자를 대상으로 BMI를 얻는 작업을 하자. 이 때 검사 대상자들의 키와 몸무게가 다음과 같이 2차원 넘파이 배열에 저장되었다고 하자.

다차원배열의 첫 행에는 대상자들의 키가 미터(m) 단위로 저장되어 있으며, 두 번째 행에는 몸무게가 킬로그램(kg) 단위로 저장되어 있다.

```
patients = np.array([[ 1.63, 1.86, 1.63, 1.76], # 환자들의 키
                    [ 76.0, 84.0, 69.0, 85.0]]) # 환자들의 몸무게
```

이 patients 다차원배열에 있는 환자들의 체질량지수(BMI)를 계산하고자 한다. BMI는 다음과 같은 식으로 구할 수 있다.

$$\text{체질량지수(BMI)} = \text{몸무게} / \text{키}^2$$

위의 정보를 바탕으로 다음과 같이 환자들의 키, 몸무게, 체질량지수를 출력하자. 마지막으로 체질량 지수가 25 미만인 값을 다시 한 번 더 출력하자.

원하는 결과

```
환자들의 키 : [1.63 1.86 1.63 1.76]
환자들의 몸무게 : [76. 84. 69. 85.]
환자들의 체질량지수 : [28.60476495 24.28026361 25.97011555 27.44059917]
체질량지수가 25미만 값 : [24.28026361]
```

힌트



patients는 2행 4열의 다차원배열이다. 이 다차원배열에서 키 행만을 출력하려면 patients[0]를 이용하면 된다. 마찬가지로 몸무게 행을 출력하려면 patients[1]을 사용한다. 이들을 별개의 변수에 저장한 다음 BMI = weights / (heights ** 2) 식을 사용하도록 한다.



LAB⁶⁻² 배열의 형태를 알아내고 슬라이싱하여 연산하기



해답 코드



```
import numpy as np

patients = np.array([[ 1.63, 1.86, 1.63, 1.76], # 환자들의 키
                    [ 76.0, 84.0, 69.0, 85.0]]) # 환자들의 몸무게

heights = patients[0]
weights = patients[1]

BMI = weights / (heights ** 2)

print('환자들의 키 :', heights)
print('환자들의 몸무게 :', weights)
print('환자들의 체질량지수 :', BMI)
print('체질량지수가 25미만 값 :', BMI[ BMI < 25 ])
```



6.9 사례 분석: 시본의 다양한 데이터 셋과 펭귄 데이터 셋



- 시본은 다음과 같은 다양한 데이터 셋을 제공함

데이터 셋 이름	설명
anscombe	Anscombe의 quartet으로, 서로 다른 네 가지의 소규모 데이터 셋을 제공한다. 이 데이터 셋은 데이터 분석에 앞서 시각화의 중요성과 특이치 값의 영향을 보여주려는 목적으로 만들어졌다.
attention	시각적 주의력이 분산된 상태와 집중 상태에서 문제를 얼마나 맞추었는가에 대한 실험 데이터이다.
car_crashes	미국의 도로 안전 데이터로, 주별 사고 통계를 담고 있다.
diamonds	약 54,000개의 다이아몬드 가격과 관련 속성을 담고 있는 데이터 셋이다.
flights	1949년부터 1960년까지의 월별 항공 승객 수 데이터 셋이다.
iris	붓꽃의 종류를 분류하기 위한 고전적인 데이터 셋이다.
penguins	팔머 군도의 펭귄 데이터 셋으로 세 가지 종류의 펭귄 종과 부리의 길이, 날개 길이 등의 측정값을 포함하고 있다.
tips	식사 금액과 팁에 대한 데이터 셋으로, 식사자 수, 요일, 시간, 흡연 유무 등의 정보를 포함한다.
titanic	타이타닉호의 승객 정보 데이터 셋으로, 생존 여부, 객실 등급, 성별, 나이 등의 정보가 포함되어 있다.

- 언급한 데이터 셋 이외에도 planets, fmri, exercise, dots, mpg 등의 다양한 데이터가 제공되고 있음



6.9 사례 분석: 시본의 다양한 데이터 셋과 펭귄 데이터 셋



- 이 데이터 셋 중에서 펭귄 데이터 셋에 대하여 살펴보자.



```
import seaborn as sns
```

```
penguins = sns.load_dataset("penguins")  
penguins.head()
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Female
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Female
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Female

- 펭귄 데이터 셋은 세 가지의 펭귄 종에 대한 부리의 길이, 날개 길이 등의 측정값을 포함하고 있음



6.9 사례 분석: 시본의 다양한 데이터 셋과 펭귄 데이터 셋



- 펭귄 데이터프레임의 각 특성과 이에 대한 설명

특성	해석	특성	해석
species	펭귄의 종으로 세 가지가 있다.	island	펭귄이 서식하는 섬
bill_length_mm	부리 길이	bill_depth_mm	부리 깊이
flipper_length_mm	날개 길이	body_mass_g	몸무게
sex	암수(Male/Female) 구분		

- `penguins.shape`으로 이 데이터 프레임의 형상을 살펴보면 모두 7개의 특성과 344개의 데이터가 있음을 확인할 수 있음

```

▶ penguins.shape
(344, 7)

```



6.10 펭귄 데이터 세트의 시각화



- 본격적인 데이터 분석과 시각화를 위해, 우선 다음과 같은 명령으로 수치 데이터의 특성에 대하여 살펴보자.

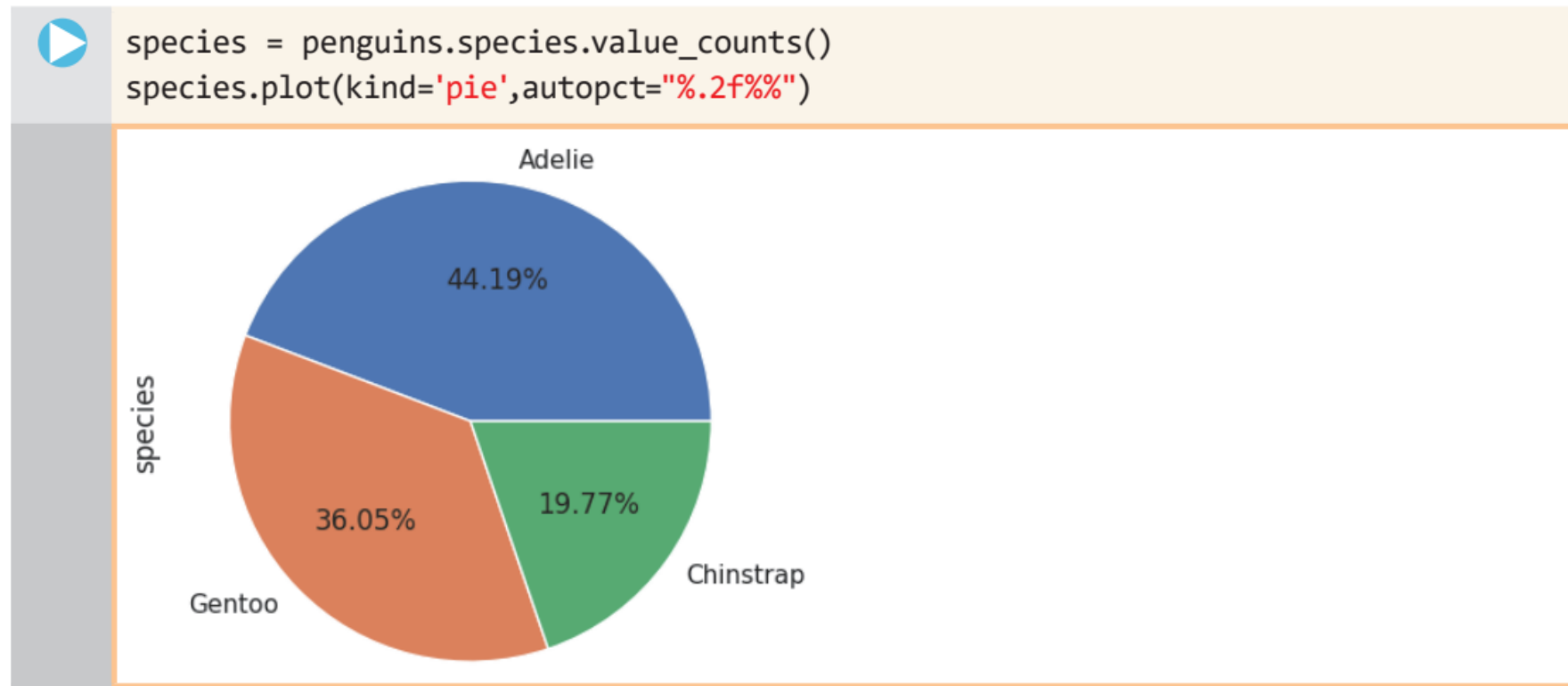
	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	342.000000	342.000000	342.000000	342.000000
mean	43.921930	17.151170	200.915205	4201.754386
std	5.459584	1.974793	14.061714	801.954536
min	32.100000	13.100000	172.000000	2700.000000
25%	39.225000	15.600000	190.000000	3550.000000
50%	44.450000	17.300000	197.000000	4050.000000
75%	48.500000	18.700000	213.000000	4750.000000
max	59.600000	21.500000	231.000000	6300.000000



6.10 펭귄 데이터 셋의 시각화



- 펭귄 데이터 셋에서 부리의 길이와 깊이, 날개의 길이, 몸무게는 수치 데이터이므로 위의 결과와 같이 평균, 표준편차, 최소값, 최대값에 대한 정보를 얻을 수 있음
- 다음으로 이 데이터 셋에 존재하는 세가지 펭귄 종의 분포를 파이 차트를 통해서 알아보자.





6.10 펭귄 데이터 셋의 시각화



- 이제 다음과 같이 펭귄의 서식지에 대한 정보를 추출해 보자.
- `penguins.island.value_counts()`를 통해서 먼저 펭귄의 서식지를 출력해 보는 것으로 시작하자.



```
penguins.island.value_counts()
```

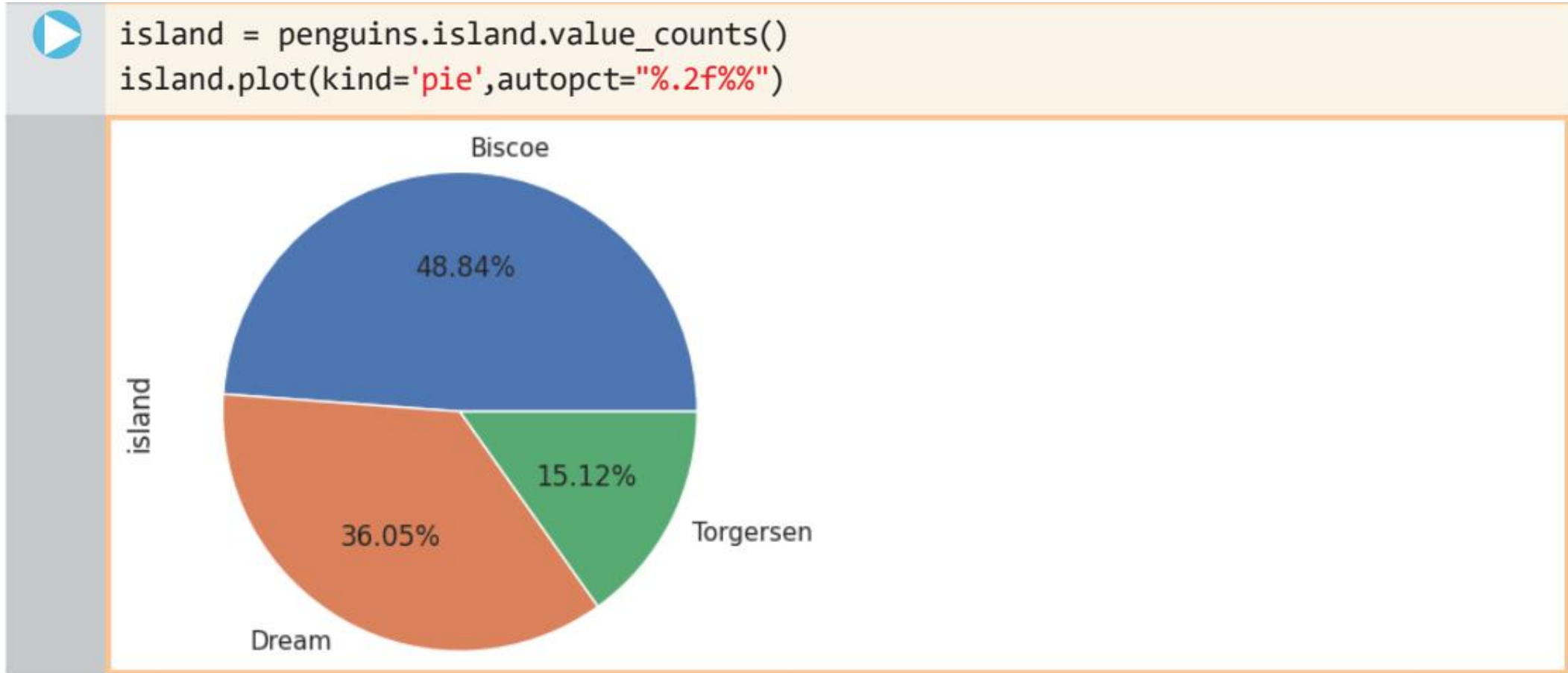
```
Biscoe      168  
Dream       124  
Torgersen   52  
Name: island, dtype: int64
```



6.10 펭귄 데이터 셋의 시각화



- 이 분포를 파이 차트를 통해 나타내면 다음과 같이 분포와 퍼센트가 나타나는 것을 볼 수 있다.





6.10 펭귄 데이터 셋의 시각화



- 펭귄 데이터 셋에 있는 펭귄은 암컷(female)과 수컷(male)으로 구별되어 있으며 각 펭귄의 암수 분포는 다음과 같은 방법으로 출력해 볼 수 있다.



```
penguins.sex.value_counts()
```

```
Male      168
```

```
Female    165
```

```
Name: sex, dtype: int64
```



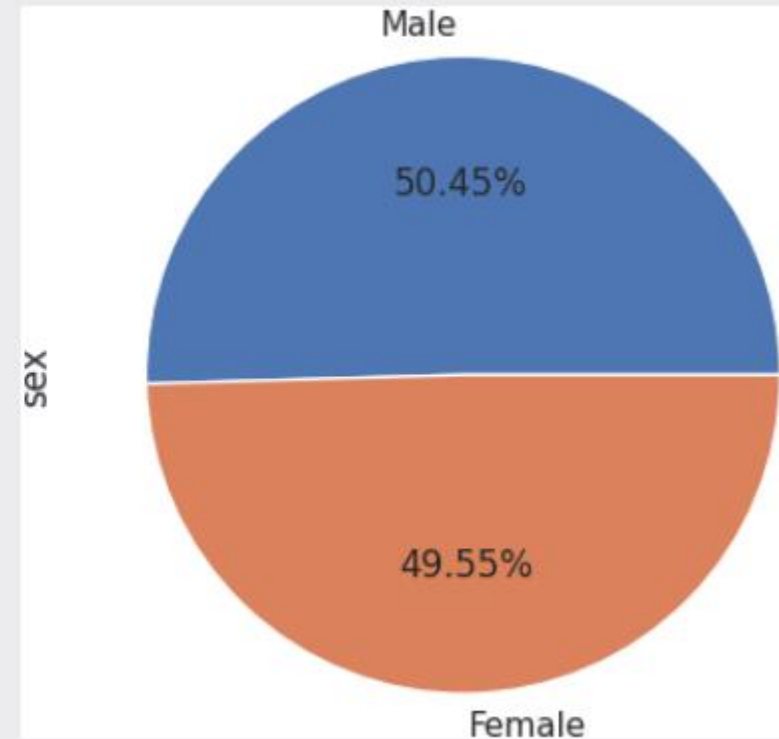
6.10 펭귄 데이터 세트의 시각화



도전문제 6.3 펭귄의 성 분포를 시각화하자

난이도 중

펭귄 데이터 세트에 있는 펭귄은 암컷(female)과 수컷(male)이 표시되어 있다. 각 펭귄의 분포를 다음과 같이 파이 차트를 이용하여 시각화 하여라.





6.11 펭귄 데이터 셋의 전체 구조를 파악하고 질의를 하자



- 펭귄 데이터 셋의 데이터프레임의 구조를 파악하는 데에 유용한 `info()` 메소드를 살펴보자.

```
▶ penguins.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 7 columns):
#   Column                Non-Null Count  Dtype
---  -
0   species               344 non-null   object
1   island                 344 non-null   object
2   bill_length_mm        342 non-null   float64
3   bill_depth_mm         342 non-null   float64
4   flipper_length_mm     342 non-null   float64
5   body_mass_g           342 non-null   float64
6   sex                   333 non-null   object
dtypes: float64(4), object(3)
memory usage: 18.9+ KB
```

- 이를 통하여 데이터프레임의 행과 열의 수, 열의 이름, 데이터 타입 및 누락된 값의 개수 등을 확인할 수 있음



6.11 펭귄 데이터 셋의 전체 구조를 파악하고 질의를 하자



데이터프레임의 질의를 통한 데이터 추출: query() 명령

- 데이터프레임에는 query()라는 메소드가 있어서 다양한 조건을 주고 질의를 통해서 데이터를 얻을 수 있음
- 예를 들어 펭귄의 종(species)중에서 젠투 펭귄(Gentoo)에 관한 데이터만 필요한 경우 다음과 같이 query("species == 'Gentoo'")라는 조건을 통해서 쉽게 젠투 펭귄에 관한 데이터를 얻을 수 있음



```
gentoo = penguins.query("species == 'Gentoo'")
gentoo.describe()
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	123.000000	123.000000	123.000000	123.000000
mean	47.504878	14.982114	217.186992	5076.016260
std	3.081857	0.981220	6.484976	504.116237
min	40.900000	13.100000	203.000000	3950.000000
25%	45.300000	14.200000	212.000000	4700.000000
50%	47.300000	15.000000	216.000000	5000.000000
75%	49.550000	15.700000	221.000000	5500.000000
max	59.600000	17.300000	231.000000	6300.000000



6.11 펭귄 데이터 세트의 전체 구조를 파악하고 질의를 하자



- 이 조건은 **species** 열의 값들 중에서 '**Gentoo**'라는 문자열을 가진 종만을 선택적으로 추출하는 기능
- 만일 몸무게가 100 이상인 조건을 주려면 `query(body_mass_g >= 100)`과 같은 식을 사용하면 됨
- `print(gentoo)` 명령을 통해 추출한 gentoo 데이터프레임의 내용을 표시해 보자.

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	\
220	Gentoo	Biscoe	46.1	13.2	211.0	
221	Gentoo	Biscoe	50.0	16.3	230.0	
..	...	...	...	...	...	
339	Gentoo	Biscoe	NaN	NaN	NaN	
342	Gentoo	Biscoe	45.2	14.8	212.0	
343	Gentoo	Biscoe	49.9	16.1	213.0	



6.11 펭귄 데이터 세트의 전체 구조를 파악하고 질의를 하자



- 이 젠투 펭귄 중에서 부리 길이가 53보다 더 긴 종을 추출하기 위해서는 gentoo 데이터프레임에 다시 질의를 주면 됨



```
gentoo.query("bill_length_mm > 53")
```

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
253	Gentoo	Biscoe	59.6	17.0	230.0	6050.0	Male
283	Gentoo	Biscoe	54.3	15.7	231.0	5650.0	Male
321	Gentoo	Biscoe	55.9	17.0	228.0	5600.0	Male
327	Gentoo	Biscoe	53.4	15.8	219.0	5500.0	Male
335	Gentoo	Biscoe	55.1	16.0	230.0	5850.0	Male



6.11 펭귄 데이터 셋의 전체 구조를 파악하고 질의를 하자



- 만일 아델리 펭귄의 데이터프레임을 만들고자 한다면 다음과 같이 질의에 들어가는 종의 이름만을 수정하면 됨



```
adelie = penguins.query("species == 'Adelie'")  
adelie.describe()
```

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
count	151.000000	151.000000	151.000000	151.000000
mean	38.791391	18.346358	189.953642	3700.662252
std	2.663405	1.216650	6.539457	458.566126
min	32.100000	15.500000	172.000000	2850.000000
25%	36.750000	17.500000	186.000000	3350.000000
50%	38.800000	18.400000	190.000000	3700.000000



6.11 펭귄 데이터 셋의 전체 구조를 파악하고 질의를 하자



도전문제 6.4 아델리 펭귄의 수컷만을 추출하자

난이도 중

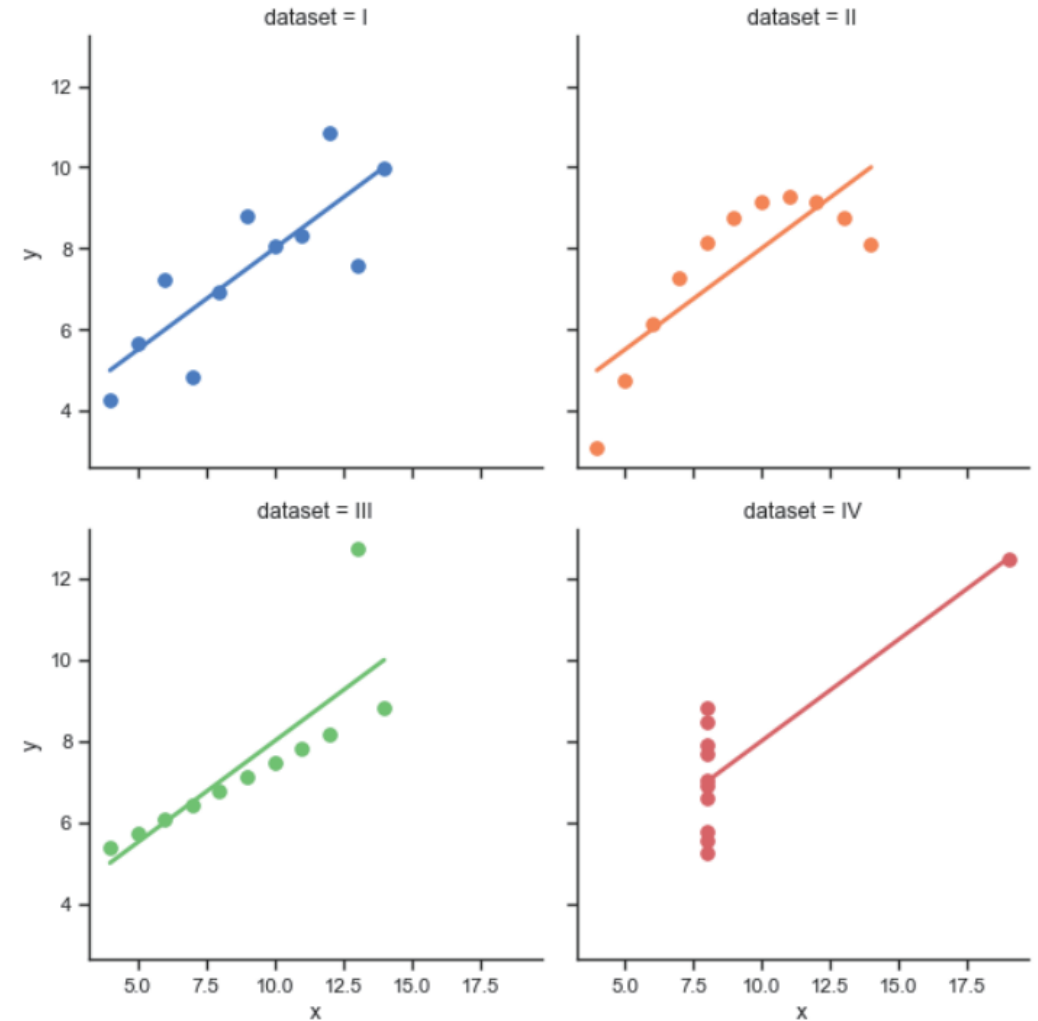
펭귄 데이터 셋에 있는 펭귄 중에서 아델리 펭귄의 수컷(male)을 추출하여 가장 먼저 나타나는 3개의 레코드를 다음과 같이 출력하여라.

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Male
5	Adelie	Torgersen	39.3	20.6	190.0	3650.0	Male
7	Adelie	Torgersen	39.2	19.6	195.0	4675.0	Male



6.12 사례 분석: Anscombe's quartet 데이터 셋

- Anscombe's quartet 데이터 셋은 다음 그림과 같은 분포를 가지는 네 가지 형태 데이터의 모음을 말함
- 이 데이터는 1973년 통계학자인 **프랜시스 앤스컴** Francis Anscombe이 데이터 분석에 앞서 시각화의 중요성과 특이치 값의 영향을 보여주기 위해 제작함
- 이 데이터 셋을 이용하여 데이터가 다양한 분포를 가질 수 있음을 살펴보고, 이들을 효과적으로 모델링 하는 방법을 알아보자.





6.12 사례 분석: Anscombe's quartet 데이터 셋



- 이 데이터 셋의 첫번째 데이터를 시각화해 보자.
- 우선 시본의 `load_dataset('anscombe')`를 사용하여 데이터를 로드한 후에 헤더 부분을 출력해 보자.



```
import seaborn as sns
```

```
anscombe = sns.load_dataset('anscombe')
```

```
print(anscombe.head(15)) # 가장 먼저 나타나는 15개의 데이터 셋
```

	dataset	x	y
0	I	10.0	8.04
1	I	8.0	6.95
2	I	13.0	7.58
3	I	9.0	8.81
4	I	11.0	8.33
5	I	14.0	9.96
6	I	6.0	7.24
7	I	4.0	4.26
8	I	12.0	10.84
9	I	7.0	4.82
10	I	5.0	5.68
11	II	10.0	9.14
12	II	8.0	8.14
13	II	13.0	8.74



6.12 사례 분석: Anscombe's quartet 데이터 셋



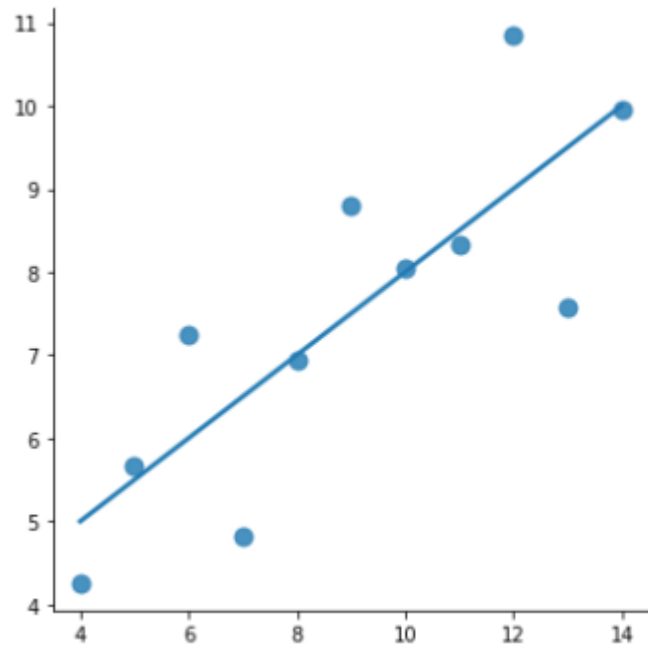
- 이 데이터의 dataset 열을 살펴보면 I, II, III, IV와 같은 로마 숫자가 나타나 있는데 이 데이터 셋들은 각각 그림의 데이터 셋과 같은 분포를 가지고 있음
- 각 데이터 셋은 모두 11개씩의 데이터로 이루어져 있으며 전체 데이터는 44개
- 이 데이터 셋의 첫 번째 데이터를 시각화하고 이 데이터의 선형 회귀 직선을 구하기 위해서는 `lplot()` 함수를 사용함
- `lplot`은 `regplot`과 유사한 선형 회귀 직선을 구하는 기능과 함께 `FacetGrid`라고 하는 화면에 보기 좋은 격자 간격을 생성하는 인터페이스를 결합한 함수



6.12 사례 분석: Anscombe's quartet 데이터 셋



```
sns.lmplot(x="x", y="y", data=anscombe.query("dataset == 'I'"),  
           ci=None, scatter_kws={"s": 80})
```

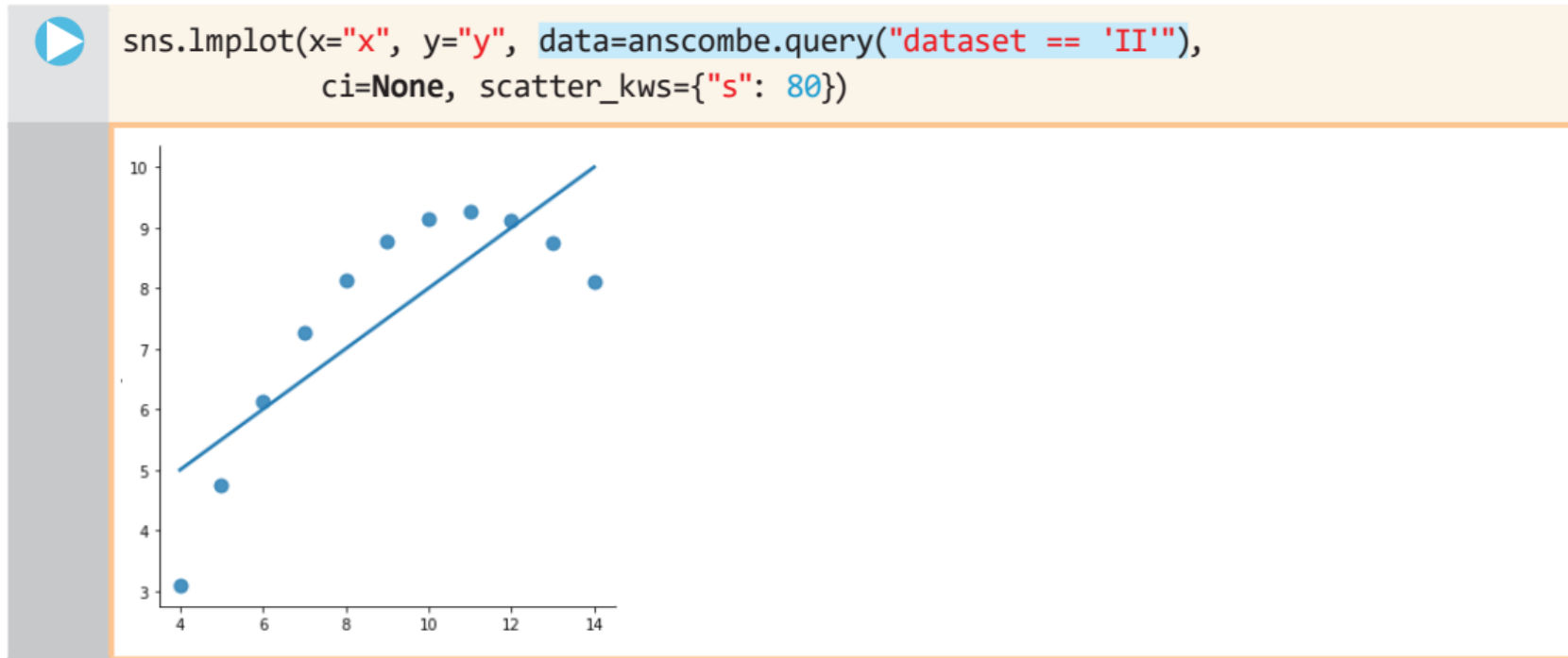




6.12 사례 분석: Anscombe's quartet 데이터 셋



- 이제 `anscombe.query("dataset == 'II'")` 명령으로 두 번째 타입의 데이터 셋을 가져와서 선형 회귀 직선으로 표현해 보자.



- 두 번째 유형의 데이터는 분포가 비선형적인데 이를 직선으로 표현하니 데이터의 분포와 선형 회귀 직선 사이에 차이가 나타난다.

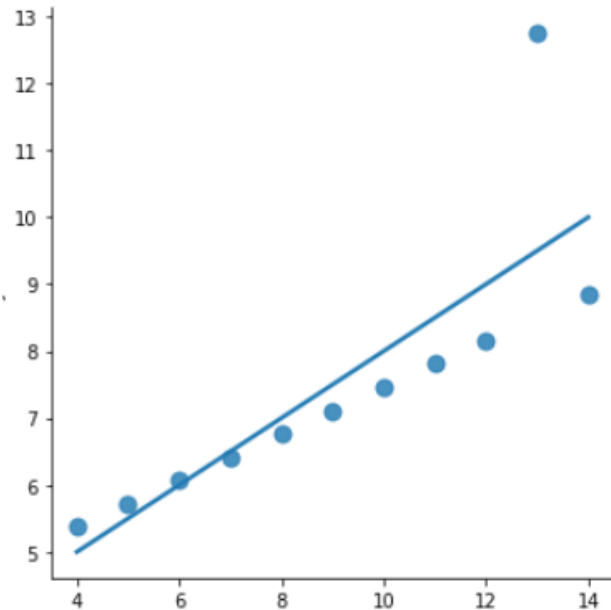


6.12 사례 분석: Anscombe's quartet 데이터 셋

- 마지막으로 세 번째 유형의 데이터 분포와 그 선형 회귀 직선을 살펴보자.



```
sns.lmplot(x="x", y="y", data=anscombe.query("dataset == 'III'"),  
           ci=None, scatter_kws={"s": 80})
```





6.13 비선형 함수를 사용하여 데이터를 설명하자



- Anscombe's quartet 데이터 셋의 분포를 알아내기 위해 데이터 셋 I과 데이터 셋 II의 평균, 표준 편차를 각각 살펴보자.



```
df = sns.load_dataset("anscombe")
df[df['dataset'] == 'I'].describe() # 첫 번째 데이터의 갯수, 평균, 표준편차 등
```

	x	y
count	11.000000	11.000000
mean	9.000000	7.500909
std	3.316625	2.031568
min	4.000000	4.260000
이하 생략		



```
df = sns.load_dataset("anscombe")
df[df['dataset'] == 'II'].describe() # 두 번째 데이터의 갯수, 평균, 표준편차 등
```

	x	y
count	11.000000	11.000000
mean	9.000000	7.500909
std	3.316625	2.031657
min	4.000000	3.100000
이하 생략		



6.13 비선형 함수를 사용하여 데이터를 설명하자



- 두 데이터를 살펴보면 데이터의 수, 평균, 표준 편차 값이 모두 같은 것을 볼 수 있음
- 나머지 데이터(데이터 셋 III, 데이터 셋 IV) 역시 데이터의 수와 평균, 표준 편차 값이 동일함
- 앞서 비선형적인 데이터의 분포를 선형회귀 직선으로 나타낼 경우 데이터의 분포와 직선이 일치하지 않는 문제점이 있었음
- 데이터 셋 II는 선형적인 분포를 갖지 않으므로 2차 함수나 3차 함수와 같은 높은 차수의 함수를 이용해야만 제대로 분포를 설명할 수 있음
- `lmpot()` 함수는 이를 위하여 `order=2`와 같은 2차 함수를 사용하는 방법을 제공함



6.13 비선형 함수를 사용하여 데이터를 설명하자



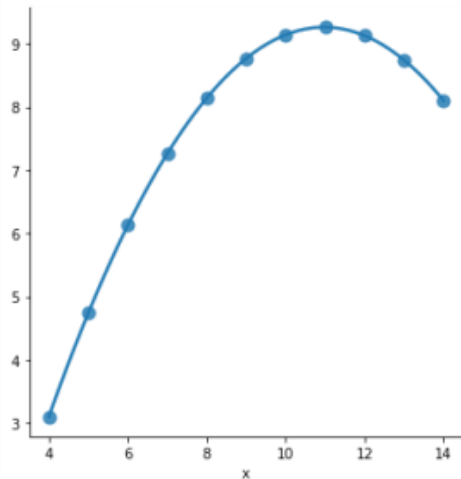
- 다음과 같이 Implot() 함수를 사용하면 이전과 달리 **2차 함수 형태의 곡선**이 나타나서 데이터의 분포를 따라 이동하는 것을 볼 수 있음



```
import seaborn as sns
```

```
anscombe = sns.load_dataset("anscombe")
```

```
sns. (x="x", y="y", data=anscombe.query("dataset == 'II'"),  
      order=2, ci=None, scatter_kws={"s": 80}) # 2차 함수로 분포를 나타냄
```





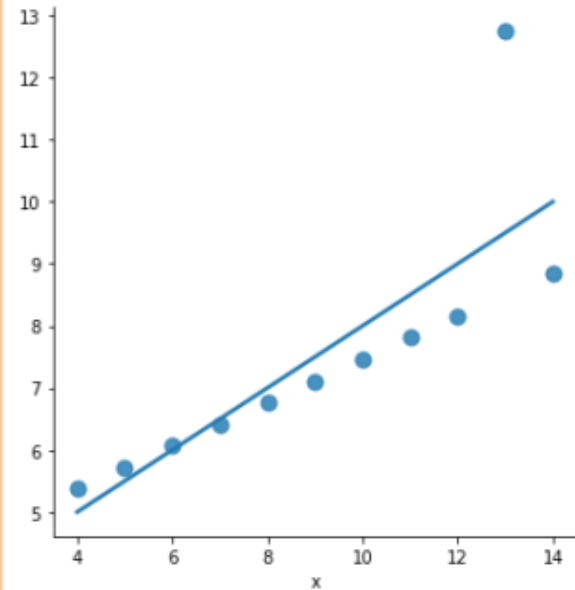
6.13 비선형 함수를 사용하여 데이터를 설명하자



- Anscombe's quartet 데이터 셋의 세 번째 유형(데이터 셋 III)은 이상치가 존재하는 경우이며, 이 이상치로 인하여 선형 회귀 직선의 형태가 왜곡되는 문제점이 있었음



```
sns.lmplot(x="x", y="y", data=anscombe.query("dataset == 'III'"),  
           ci=None, scatter_kws={"s": 80})
```





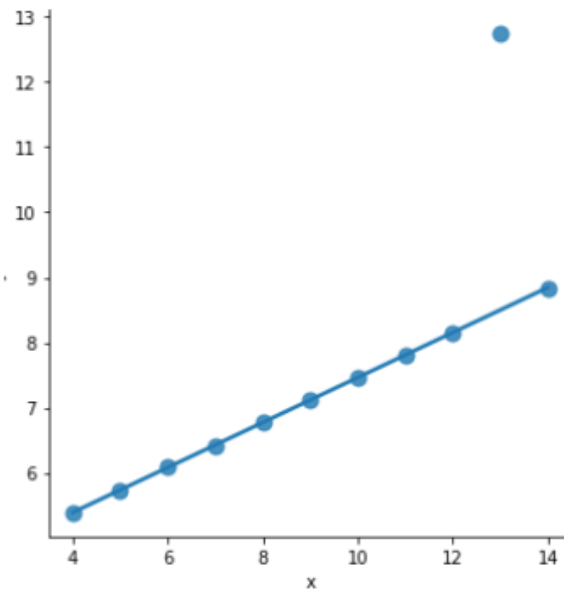
6.13 비선형 함수를 사용하여 데이터를 설명하자



- 이러한 문제점을 해결하기 위하여 `lplot()` 함수는 `robust=True`라는 키워드 인자를 제공함
- 이 키워드 인자는 이상치에 덜 민감한 강건한 모델을 만드는 방법



```
sns.lplot(x="x", y="y", data=anscombe.query("dataset == 'III'"),  
          robust=True, ci=None, scatter_kws={"s": 80})
```





6.13 비선형 함수를 사용하여 데이터를 설명하자



- 이 밖에도 `lplot()` 함수는 뒤에 배열 로지스틱 회귀를 모델링하는 기능과 같은 **고급 기능을 제공함**



도전문제 6.5 Anscombe's quartet 데이터 셋 III, IV

난이도 중

시본 라이브러리의 Anscombe's quartet 데이터 셋 III과 IV에 대하여 `describe()` 함수를 사용하여 데이터의 수와 평균, 표준편차, 최소값, 최대값을 각각 조사하여 출력하여라.



6.14 사례 분석: 항공기 이용 승객 데이터 셋



- 시본에서 제공하는 또 다른 데이터 셋으로는 항공기를 이용하는 승객 데이터인 flights 데이터 셋이 있으며, 1949년부터 1960년까지의 월간 항공기 이용자의 수를 제공하고 있음 (단위: 천 명)

```
▶ import seaborn as sns

# seaborn에서 제공하는 flights 데이터 셋을 로딩함
flights = sns.load_dataset('flights')
flights.shape # 데이터의 형상을 살펴보자
```

```
(144, 3)
```

```
▶ flights.head() # 최초 5개의 데이터를 살펴보자
```

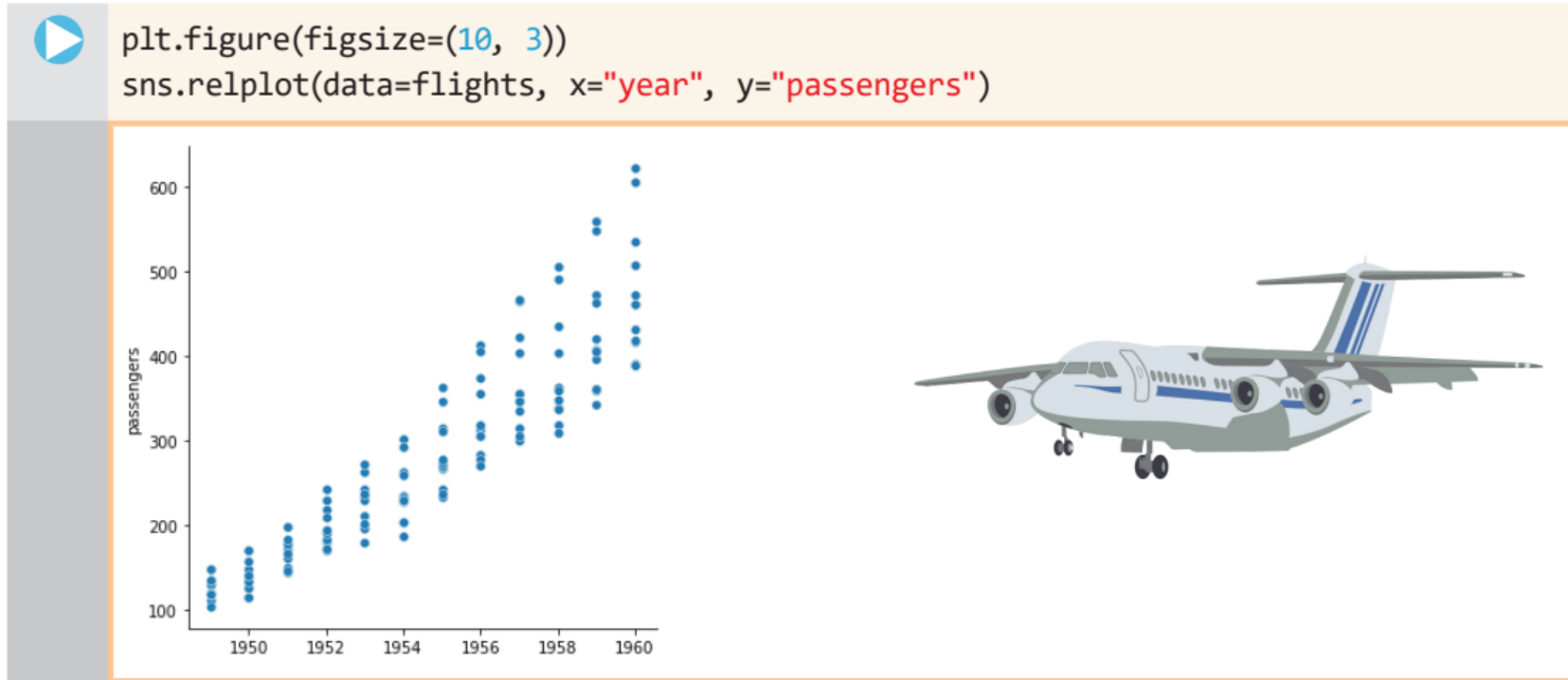
	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121



6.14 사례 분석: 항공기 이용 승객 데이터 셋



- 전체 데이터에 대한 통찰을 얻기 위하여 다음과 같은 코드를 사용하여 시각화시켜 보자.



- x축의 값은 년도(year) 값이므로 이산적으로 표시되는 것을 볼 수 있으며, y축의 값은 승객의 수를 나타내는 것을 볼 수 있음

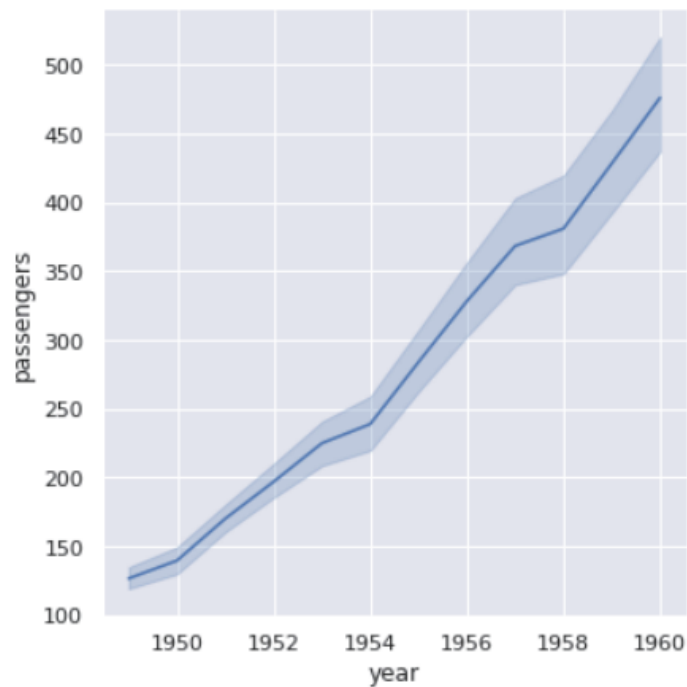


6.14 사례 분석: 항공기 이용 승객 데이터 셋

- 위의 그래프가 이산적인 점의 형태로 나타나 보기에 불편하다면, 시본 라이브러리의 `relplot(..., kind = "line")` 함수를 사용하여 보완할 수 있음



```
plt.figure(figsize=(10, 3))  
sns.relplot(data=flights, x="year", y="passengers", kind="line")
```





6.14 사례 분석: 항공기 이용 승객 데이터 셋



- kind = "line"인 경우 산점도가 아닌 직선의 형태로 그려주기 때문에 데이터의 구조를 이해하는 것이 더 편해짐
- 이때 나타나는 굵은 실선은 월평균 이용자 수이며, 진하게 나타나는 내부의 면은 최소 이용자와 최대 이용자 사이의 범위를 나타냄



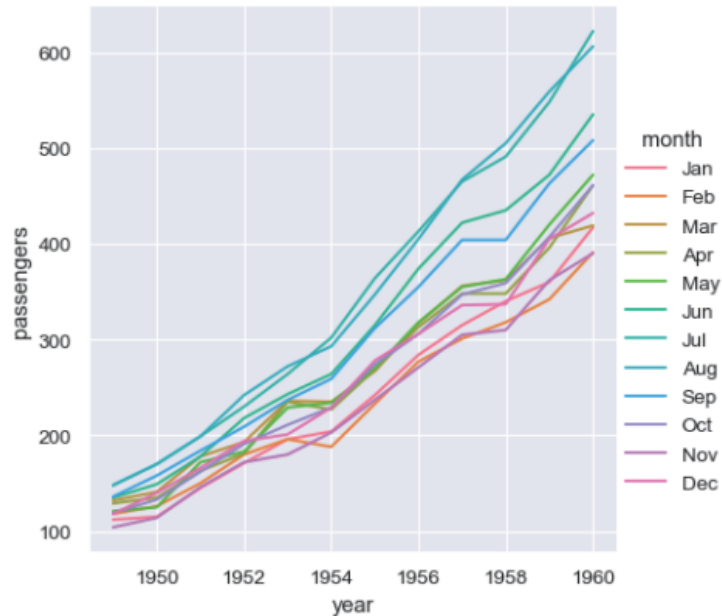
6.14 사례 분석: 항공기 이용 승객 데이터 셋



- 이 데이터 셋은 다음과 같은 방법으로 시각화 가능



```
plt.figure(figsize=(10, 3))
sns.relplot(data=flights, x="year", y="passengers", hue="month", kind="line")
```



- 이 시각화 방법에서 `hue="month"`는 month라는 열에서 동일한 값을 가지는 데이터를 같은 색으로 나타내어 주는 역할



6.15 항공기 이용 승객 데이터 셋을 고쳐보자



- 시본의 항공기 이용 승객 데이터 셋을 수정해서 테이블을 새롭게 만들어 보자.
- 만들어볼 테이블의 행은 1949년부터 1960년도까지이며 열은 1월, 2월, 3월, ... 이 되도록 할 것이다.
- 이를 위해 먼저 flights의 자료형을 확인해 보자.



```
import seaborn as sns
```

```
# seaborn에서 제공하는 flights 데이터 셋을 로딩함
```

```
flights = sns.load_dataset('flights')
```

```
print('type(flights) =', type(flights))
```

```
type(flights) = <class 'pandas.core.frame.DataFrame'>
```



6.15 항공기 이용 승객 데이터 셋을 고쳐보자



- flights의 자료형은 판다스 데이터프레임 자료형이므로, pivot() 메소드를 사용해서 인덱스는 연도(year), 열은 달(month), 테이블의 값은 승객(passengers)이 되도록 구조를 수정할 수 있음



```
flights_wide = flights.pivot(index="year", columns="month", values="passengers")  
flights_wide.head()
```

month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
year												
1949	112	118	132	129	121	135	148	148	136	119	104	118
1950	115	126	141	135	125	149	170	170	158	133	114	140
1951	145	150	178	163	172	178	199	199	184	162	146	166
1952	171	180	193	181	183	218	230	242	209	191	172	194
1953	196	196	236	235	229	243	264	272	237	211	180	201

- 이를 위하여 pivot() 메소드의 키워드 인자를 index="year", columns="month", values="passengers"로 설정함



6.15 항공기 이용 승객 데이터 셋을 고쳐보자



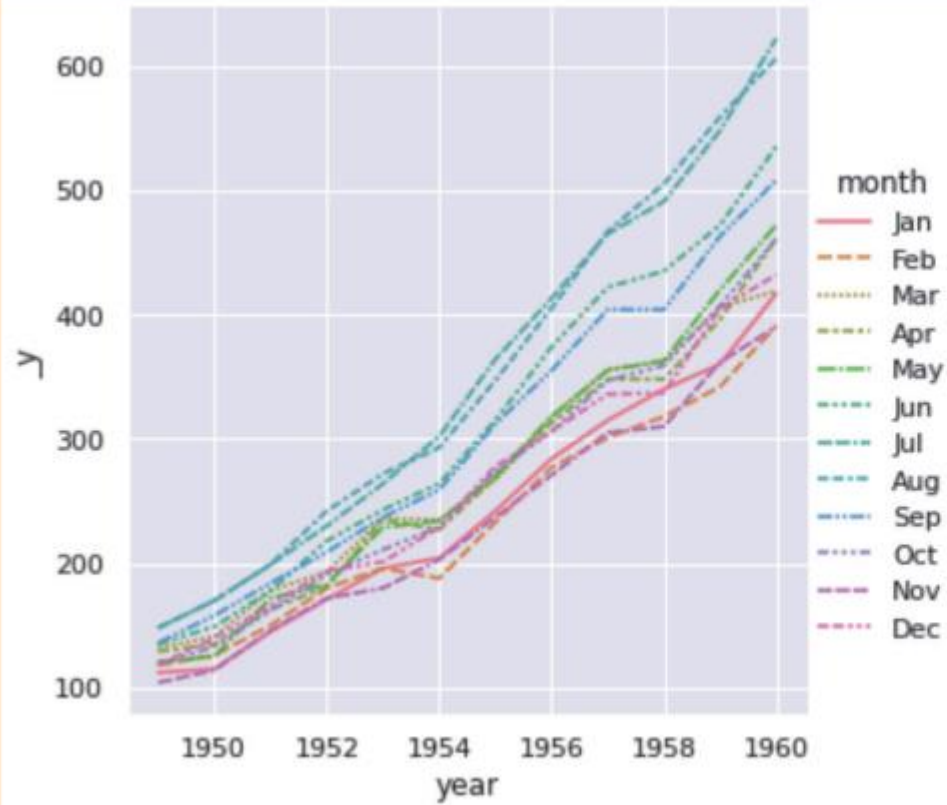
- 이제 flights_wide라는 새로운 데이터프레임을 relplot()으로 시각화시켜보자.
- 시각화 결과는 다음과 같이 월별 이용객이 색상을 가진 실선과 점선으로 나타나며 가로축은 연도, 세로축은 이용객 수로 나타남



6.15 항공기 이용 승객 데이터 셋을 고쳐보자



```
sns.relplot(data=flights_wide, kind="line")
```





6.15 항공기 이용 승객 데이터 셋을 고쳐보자

- flights_wide 데이터 셋의 transpose()를 호출하여 head 만을 출력해보면 다음과 같이 행에는 월, 열에는 연도가 나타나게 됨
- 이렇게 전치를 시킬경우 위의 정보를 연도별로 표시할 수 있음



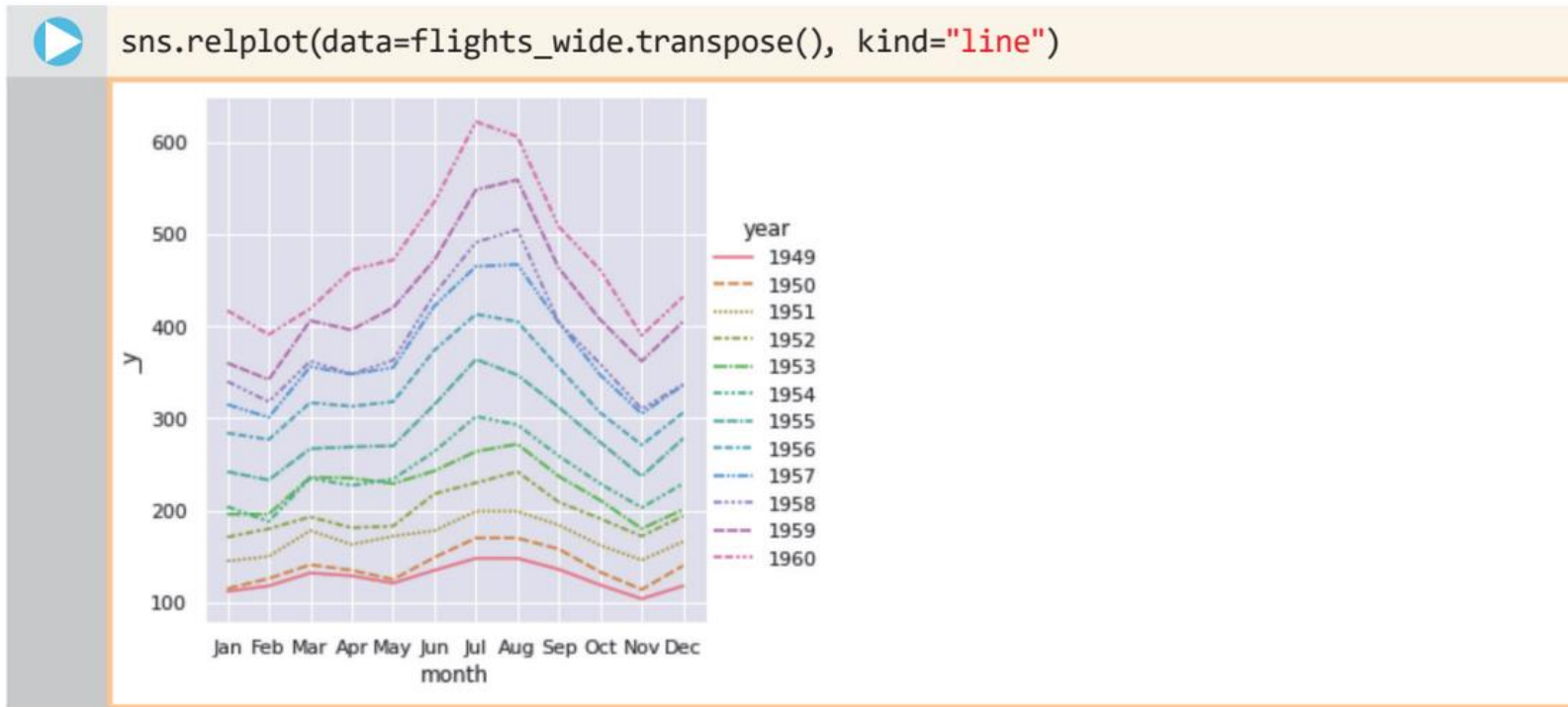
flights_wide.transpose().head()

year	1949	1950	1951	1952	1953	1954	1955	1956	1957	1958	1959	1960
month												
Jan	112	115	145	171	196	204	242	284	315	340	360	417
Feb	118	126	150	180	196	188	233	277	301	318	342	391
Mar	132	141	178	193	236	235	267	317	356	362	406	419
Apr	129	135	163	181	235	227	269	313	348	348	396	461
May	121	125	172	183	229	234	270	318	355	363	420	472



6.15 항공기 이용 승객 데이터 셋을 고쳐보자

- 이제 `transpose()`를 통해서 행과 열을 전치시킨 데이터프레임을 다음과 같이 `relplot()`으로 시각화 하도록 하자.



- 결과는 위와 같이 1949년부터 1960년도까지의 연도가 선으로 표시되며, 1월, 2월, 3월, ... 과 같은 월이 수평축의 값으로 나타난다.



6.16 히트맵을 알아보자



- **히트맵**은 열을 뜻하는 히트(heat)와 지도를 뜻하는 맵(map)을 결합시킨 단어로, 색상으로 표현할 수 있는 다양한 정보를 일정한 이미지 위에 열 분포 형태의 비주얼한 그래픽으로 출력하는 기법
- 앞서 살펴본 tips 데이터에 있는 수치값 변수들 중에서 total_bill, tip, size라는 변수들간의 상관행렬을 수치값으로 나타내면 다음과 같음



```
import seaborn as sns
```

```
tips = sns.load_dataset("tips")  
tips.corr(numeric_only=True)
```

	total_bill	tip	size
total_bill	1.000000	0.675734	0.598315
tip	0.675734	1.000000	0.489299
size	0.598315	0.489299	1.000000



6.16 히트맵을 알아보자



- 이와 같이 수치값으로 상관계수를 나타낼 수도 있으나 데이터의 크기가 커지게 되면 이 상관계수를 한 눈에 파악하기에 불편해짐
- 따라서 이를 히트맵으로 나타내어 파악하기 쉽게 만듦
- 히트맵은 `sns.heatmap()`이라는 함수로 나타낼 수 있으며 `cmap`이라는 키워드로 히트맵의 색상지를 선택할 수 있음
- 다음 코드의 실행 결과에서 오른쪽 막대 색상그래프는 범례에 해당하는 것으로, 노란색이 1에 가깝고, 어두운 청색이 0.5에 가까운 값을 나타냄



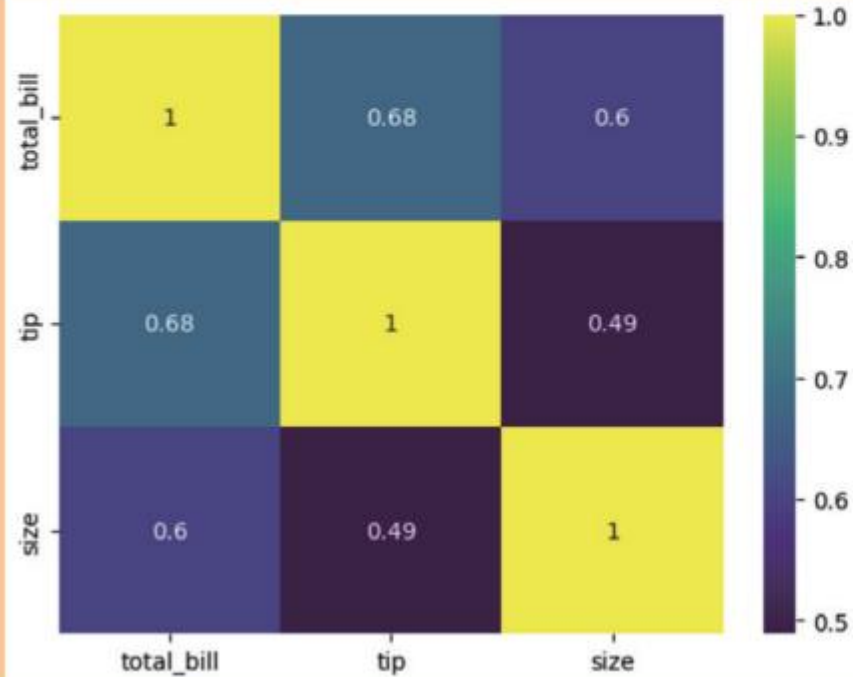
6.16 히트맵을 알아보자



```
import seaborn as sns
```

```
tips = sns.load_dataset("tips")
```

```
sns.heatmap(tips.corr(numeric_only=True), annot = True, cmap = 'viridis')
```





6.16 히트맵을 알아보자



flights 데이터 셋의 히트맵 표현

- 연도별, 월별 항공기 이용 승객에 대한 정보를 담고있는 flights 데이터 셋을 히트맵으로 나타내어 보자.



```
import seaborn as sns
import matplotlib.pyplot as plt

flights = sns.load_dataset('flights')
flights_psng = flights.pivot(index="month", columns=["year"], values="passengers")
plt.title("Flights Heatmap")
sns.heatmap(flights_psng, annot=True, fmt="d", linewidths=1)
```

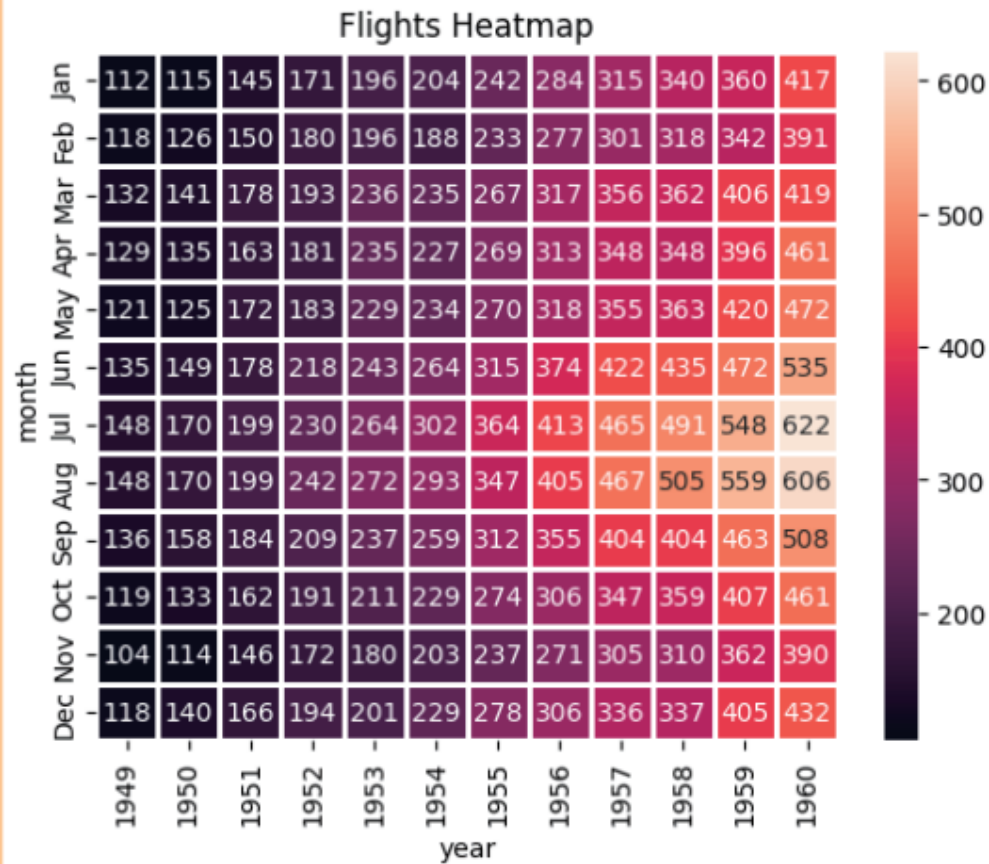
- `pivot()` 메소드는 데이터프레임의 행과 열, 값 등과 같은 구조를 변경시키는 메소드로 자세한 기능은 추후에 다룰 것이다.



6.16 히트맵을 알아보자



- 실행 결과





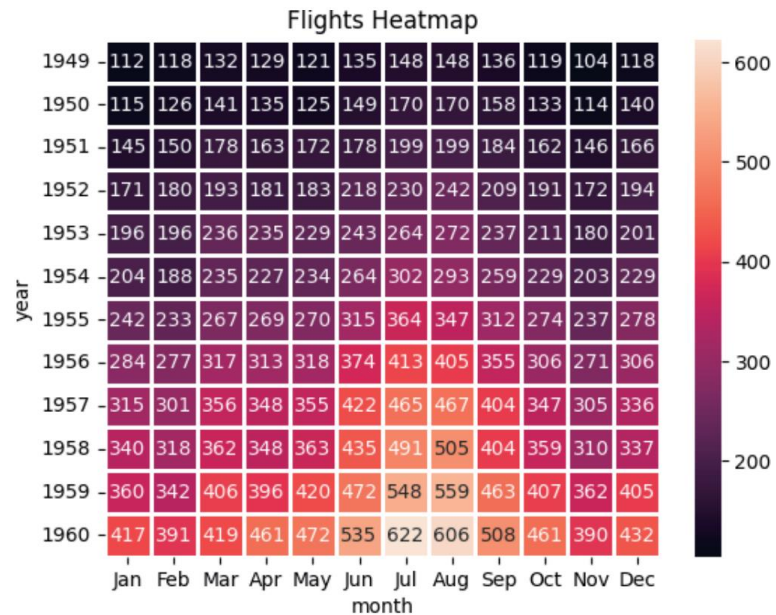
6.16 히트맵을 알아보자



- 이제 pivot() 메소드의 인자에서 행의 값을 year로, 열의 값을 month로 수정해 보자.

```
...
flights_psng = flights.pivot(index="month", columns=["year"], values="passengers")
...
```

- 결과는 다음과 같이 전치행렬에 대한 히트맵이 나오는 것을 확인할 수 있다.





- 두 개의 변수들이 함께 변화하는 관계를 나타내는 통계적 척도를 상관관계라고 한다.
- 이 변수들 사이의 상관관계 정도를 나타내는 수치값을 상관계수라고 한다.
- 한 변수가 다른 변수의 직접적인 원인이 되는 관계가 인과관계이다.
- 이 인과관계는 일반적으로 시간적 선후 관계를 포함한다.
- 시본 라이브러리는 맷플롯립을 기반으로 만들어졌으며, 높은 수준의 인터페이스를 제공한다.
- 사용자들은 맷플롯립 보다 편리하게 데이터를 분석하고 시각화할 수 있다.
- 시본 라이브러리는 여러 종류의 데이터 집합을 제공하고 있어서 데이터 분석에 좀 더 적합하다.
- 시각화 기법 중에서 열지도는 이미지 위에 열분포 형태의 그래픽으로 정보를 나타내는 방법이다.



시본 라이브러리는 맷플롯립에 기반하고 있지만 좀 더 높은 수준의 기능을
제공합니다.
특히 여러 종류의 풍부한 데이터 집합을 제공하고 있습니다.



그러면 시본과 맷플롯립은 함께 사용할 수 있겠네요.





핵심정리



그래요. 맷플롯립도 좋은 시각화 도구이기는 하지만 시본은 보다 다양하고 풍부한 기능을 제공하지요.
무엇보다 유려한 시각화 기능이 큰 장점입니다.



그것 말고도 장점이 또 있나요?

항공기 이용자 수에 대한 데이터인 flights 데이터 셋, 식당의 팁과 손님과의 관계 데이터 셋, 붓꽃 데이터 셋과 같은 풍부한 예제 데이터 집합이 제공됩니다.
이 데이터 셋을 사용하여 시본의 여러 가지 기능을 쉽게 테스트할 수 있습니다.





Questions?